

Содержание

Введение	5
1 Обзор аналогичных решений и постановка задачи	6
1.1 Обзор аналогов	6
1.1.1 Интернет-ресурс «Canva»	6
1.1.2 Интернет-ресурс «PhotoCollage»	7
1.1.3 Интернет-ресурс «Progif»	9
1.2 Постановка задачи	10
1.3 Выводы по разделу	11
2 Проектирование web-приложения	12
2.1 Функциональные возможности web-приложения	12
2.2 Проектирование базы данных	14
2.3 Архитектура web-приложения	18
2.4 Выводы по разделу	19
3 Реализация web-приложения	20
3.1 Программная платформа Node.js	20
3.2 Система управления базами данных PostgreSQL	21
3.3 Реализация базы данных	21
3.4 Программные библиотеки	24
3.5 Паттерны	25
3.6 Описание серверной части	25
3.7 Реализация функций пользователя с ролью «Гость»	27
3.7.1 Аутентификация	27
3.7.2 Регистрация	28
3.7.3 Проверка доступности сервиса	29
3.8 Реализация функций пользователя с ролью «Клиент»	29
3.8.1 Создание проекта коллажа	29
3.8.2 Сохранение изменений проекта (обновление коллажа)	30
3.8.3 Удаление проекта	30
3.8.4 Редактирование профиля	30
3.8.5 Получение списка шаблонов	31
3.8.6 Создание пользовательского шаблона	31
3.8.7 Удаление пользовательского шаблона	31
3.8.8 Загрузка файла на сервер и пополнение медиатеки	32
3.9 Структура клиентской части	33
3.10 Настройка конфигурации Docker	34
3.11 Настройка конфигурации Nginx	35
3.12 Выводы по разделу	35
4 Тестирование web-приложения	36
4.1 Функциональное тестирование	36
4.2 Автоматизированное тестирование	37
4.3 Нагрузочное тестирование	39
4.4 Выводы по разделу	40
5 Руководство пользователя	41

5.1 Руководство для роли «Гость»	41
5.1.1 Регистрация	41
5.1.2 Доступ к функциям без авторизации	41
5.2 Руководство для роли «Клиент»	42
5.2.1 Аутентификация	43
5.2.2 Создание коллажа и редактор.....	43
5.2.3 Шаблоны, медиатека и экспорт.....	44
5.2.4 Редактирование профиля	44
5.2.5 Выход из системы.....	45
5.3 Выводы по разделу	45
6 Руководство программиста.....	46
6.1 Требования к окружению.....	46
6.2 Переменные окружения	46
6.3 Запуск без Docker.....	47
6.4 Запуск с Docker Compose	48
6.5 Проверка работоспособности	49
6.6 Выводы по разделу	50
Заключение	51
Список используемых источников.....	52
ПРИЛОЖЕНИЕ А	53
ПРИЛОЖЕНИЕ Б	55
ПРИЛОЖЕНИЕ В	58
ПРИЛОЖЕНИЕ Г	62
ПРИЛОЖЕНИЕ Д	65
ПРИЛОЖЕНИЕ Е	68

Введение

Развитие web-технологий и браузерных API сделало возможным полноценную графическую обработку изображений на стороне клиента без установки десктопных редакторов. Распространённый класс решений – онлайн-редакторы и конструкторы коллажей: они сочетают работу с растровой графикой, слоями и текстом, интеграцию с внешними источниками изображений и сохранение проектов на сервере. Для пользователя такие сервисы снижают порог входа при создании визуального контента; для разработчика важны модульная архитектура, разделение клиента и API и предсказуемое хранение данных о проектах и медиафайлах.

Целью курсового проекта является создание интернет-сервиса «Конструктор коллажей» – web-приложения для составления, редактирования и экспорта коллажей с возможностью сохранения пользовательских проектов и работы с шаблонами и медиаконтентом.

Для достижения цели необходимо решить следующие задачи:

- проанализировать существующие аналоги сервисов создания коллажей и редакторов графики в браузере;
- спроектировать архитектуру web-приложения (клиент, сервер API, хранилище данных, развёртывание);
- разработать приложение в соответствии с принятой архитектурой;
- провести тестирование и отладку системы;
- подготовить руководство пользователя и руководство программиста.

Целевая аудитория сервиса – пользователи, которым нужен доступный инструмент для быстрого создания коллажей: студенты, специалисты по визуальным материалам и широкий круг пользователей соцсетей; основная группа – зарегистрированные пользователи, работающие с проектами в редакторе.

В качестве программной платформы для разрабатываемого приложения выбраны клиент на базе React 18.2 [1] со сборщиком Vite 5.1 [2], сервер на Node.js 20 [3] с использованием Express 4.18 [4], СУБД PostgreSQL 15 [5] и доступ к данным через ORM Prisma 5.10 [6]; развёртывание предусмотрено с применением Docker [7] и Docker Compose [8], обращение пользователя к системе – через Nginx [9] в качестве обратного прокси с поддержкой HTTPS [10].

1 Обзор аналогичных решений и постановка задачи

1.1 Обзор аналогов

1.1.1 Интернет-ресурс «Canva»

Одним из наиболее известных зарубежных аналогов в области браузерной графики и коллажей является сервис Canva [11]. Главная страница представлена на рисунке 1.1.

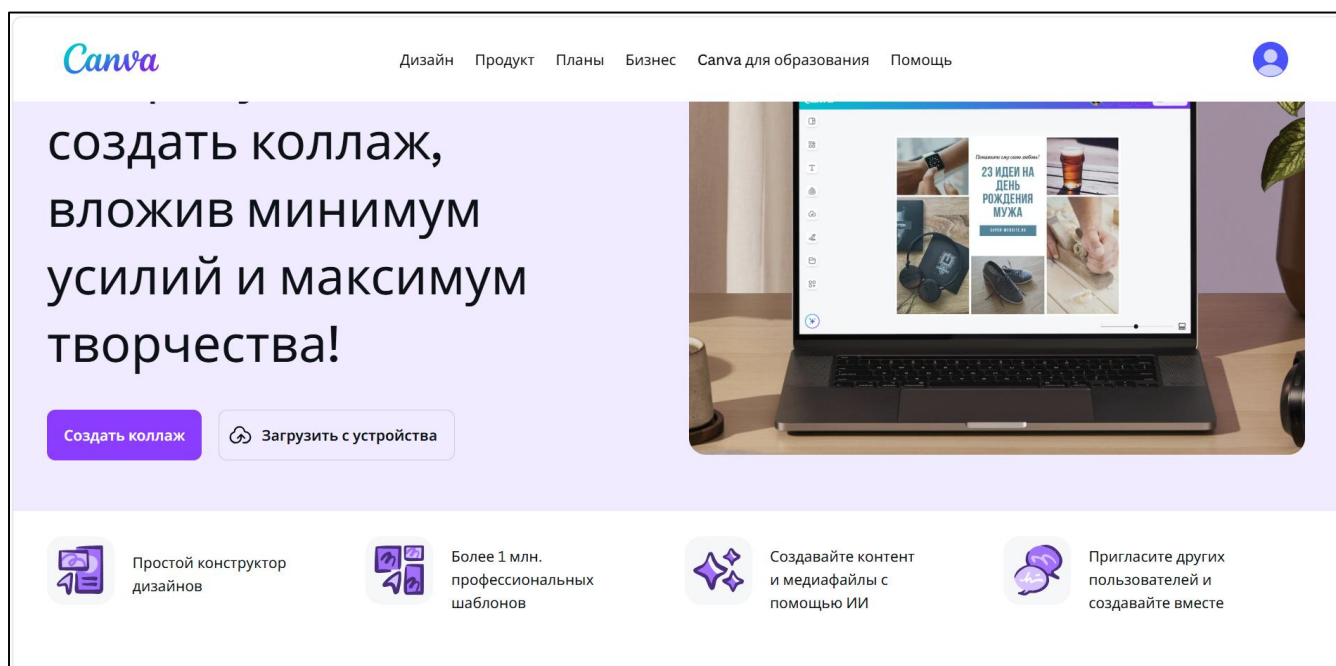


Рисунок 1.1 – Главная страница web-сайта «Canva»

По критерию доступа и учётных записей типичный сценарий предполагает регистрацию и облачное хранение проектов; без входа возможности ограничены. Знакомство с сервисом и выход на первый результат у нового пользователя организованы на высоком уровне: большой каталог шаблонов и сценариев «открыть – изменить текст – экспортировать», что снижает порог входа для непрофессионала. Организация экрана редактора выстроена вокруг холста и контекстных панелей; плотность элементов высокая, но структура выучиваемая за счёт повторяемых паттернов. Работа с композицией – свободная, со слоями, группировкой, большим набором объектов; это ближе всего к заявленным требованиям к «полноценному» редактору коллажа в браузере. Шаблоны и пресеты – ключевое преимущество сервиса. Источники изображений включают встроенные медиатеки и платный/ограниченный доступ к расширенным ресурсам. Текст и декоративные элементы представлены широко. Обработка изображений и эффекты реализованы глубже, чем у простых коллажных утилит. Сохранение и облако – сильная сторона: проекты доступны с разных устройств.

С точки зрения приватности пользователь загружает материалы в облако платформы, что для корпоративных или чувствительных данных может быть недостатком. Пример страницы создания коллажа представлен на рисунке 1.2.

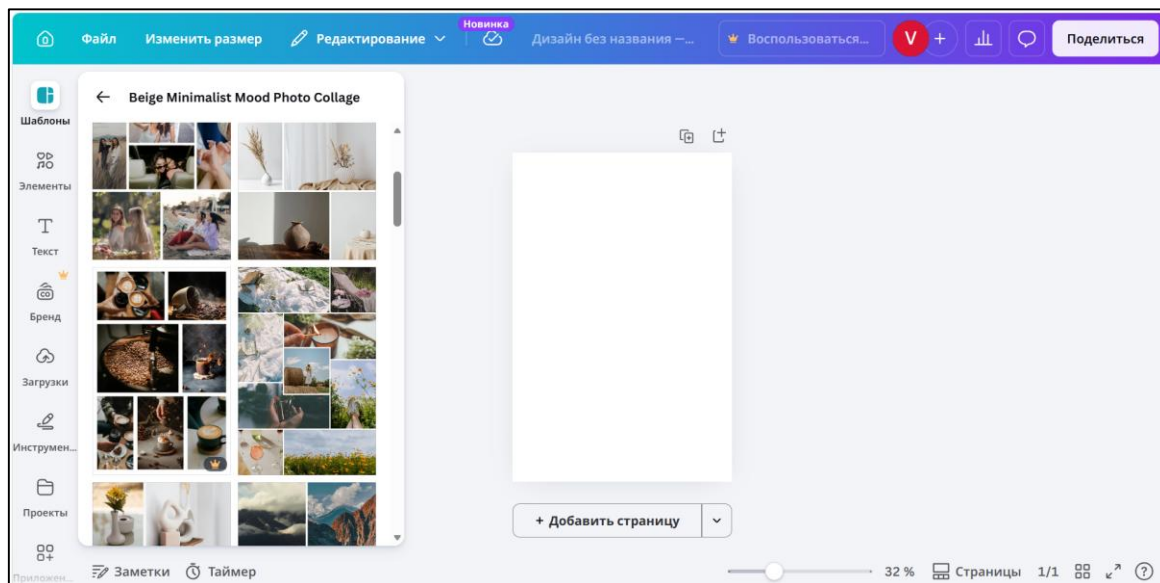


Рисунок 1.2 – Страница создания коллажа

Также реализована функция экспорта готовых работ в различных форматах. Пример экспорта представлен на рисунке 1.3.

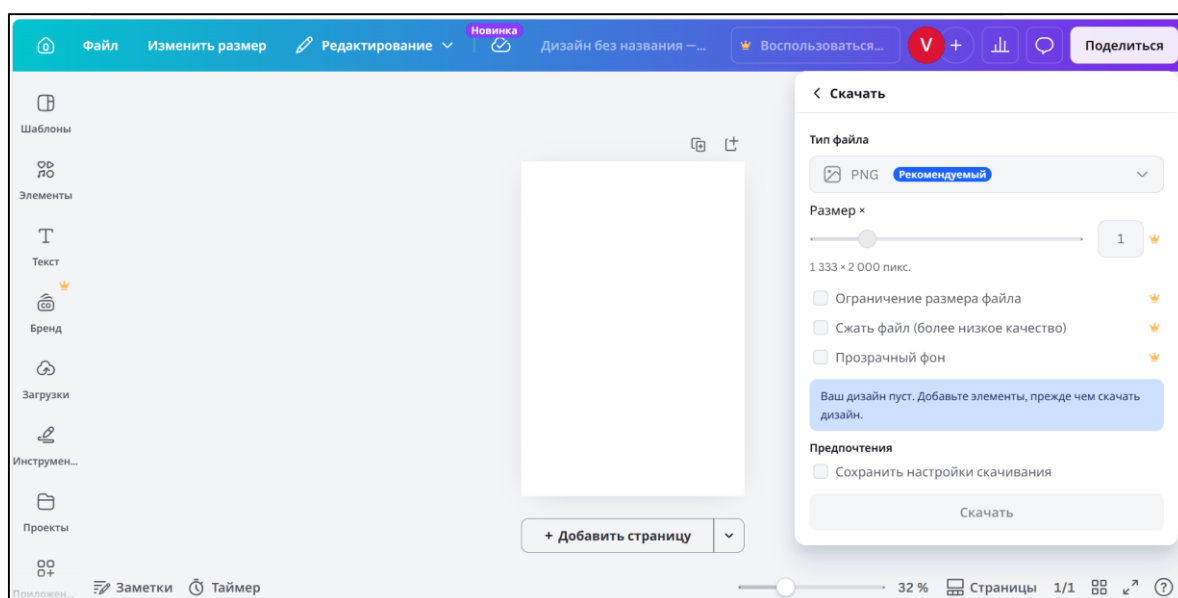


Рисунок 1.3 – Экспорт готового коллажа

К преимуществам можно отнести широкий функционал, большое количество шаблонов и удобный интерфейс. К недостаткам – ограниченный доступ к части функционала без платной подписки.

1.1.2 Интернет-ресурс «PhotoCollage»

Сервис «PhotoCollage» [12] рассчитан на пользователя, которому нужен готовый коллаж из нескольких фотографий без длительной настройки интерфейса: достаточно выбрать схему расположения кадров и подставить снимки, после чего композиция собирается за короткое время. Число доступных параметров созда-

тельно ограничено, чтобы не отвлекать от основной задачи. Такой подход удобен для нерегулярного использования и для людей без опыта работы в «тяжёлых» графических редакторах, когда важнее предсказуемый результат, чем свобода композиции на уровне профессионального дизайна. Пример того, как выглядит главная страница сервиса и как организован вход в сценарий создания коллажа, приведён на рисунке 1.4.

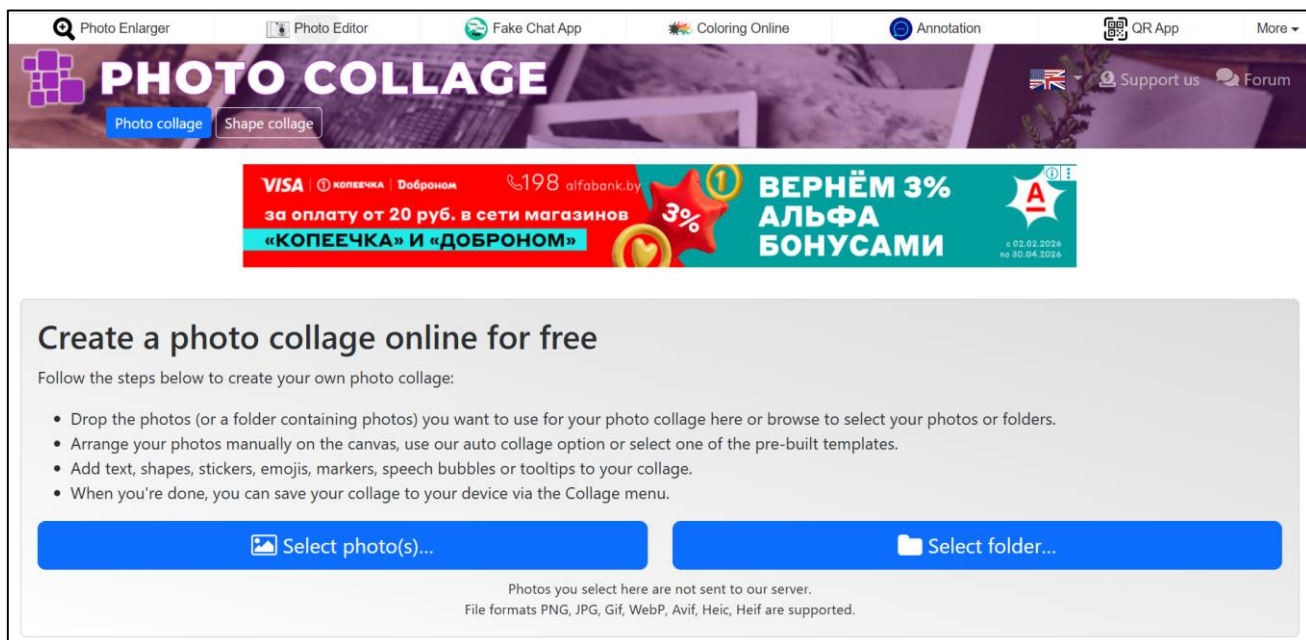


Рисунок 1.4 – Главная страница web-сайта «PhotoCollage»

По заявлению разработчиков изображения для композиции не отправляются на их сервер, обработка выполняется на стороне клиента, что положительно сказывается на критерии «приватность и объём передачи данных», но ограничивает сценарии, где нужна единая медиатека пользователя на сервере. Первый сценарий для нового пользователя предельно прост: выбор схемы сетки, загрузка снимков, автоматическое заполнение ячеек.

Интерфейс обычно минималистичен (сетка, предпросмотр). Композиция привязана к ячейкам; свободный холст, произвольные слои и глубина правок ниже, чем у Canva и ниже, чем у целевого редактора на Fabric.js. «Шаблоны» здесь – в основном варианты сеток и раскладок, не библиотека многостраничных макетов. Картинки – с устройства; стоки и развитая медиатека не в центре концепции. Подписи и декор – упрощённо.

Облачная галерея проектов не является опорой сервиса: упор на файл после сборки. За простоту платят ограниченной ручной настройкой и почти отсутствием полноценной работы со слоями и сложной графикой. Экспорт ориентирован на получение файла после сборки композиции. При этом возможности ручной настройки ограничены, что существенно упрощает процесс работы, но одновременно снижает гибкость редактирования. В отличие от более функциональных решений, данный сервис практически не предоставляет инструментов для работы со слоями или сложными графическими элементами. Пример создания коллажа представлен на рисунке 1.5.

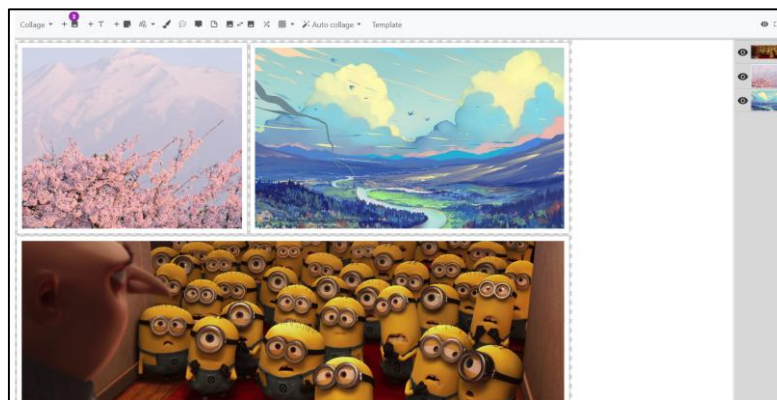


Рисунок 1.5 – Создание коллажа

Преимуществом данного сервиса является простота использования и высокая скорость формирования коллажей, что делает его удобным для пользователей, которым необходим быстрый результат без детальной настройки. В то же время ограниченность инструментов и отсутствие расширенных возможностей редактирования делают его менее подходящим для более сложных задач.

1.1.3 Интернет-ресурс «Progif»

Progif [13] – отечественный онлайн-сервис, в котором основной упор сделан на готовые макеты коллажей, базовую обработку снимков фильтрами и оформление подписями: пользователю не нужно собирать композицию «с нуля», если его устраивает один из предложенных шаблонов. Пример того, как выглядит главная страница и как из неё можно перейти к выбору макета, приведён на рисунке 1.6.

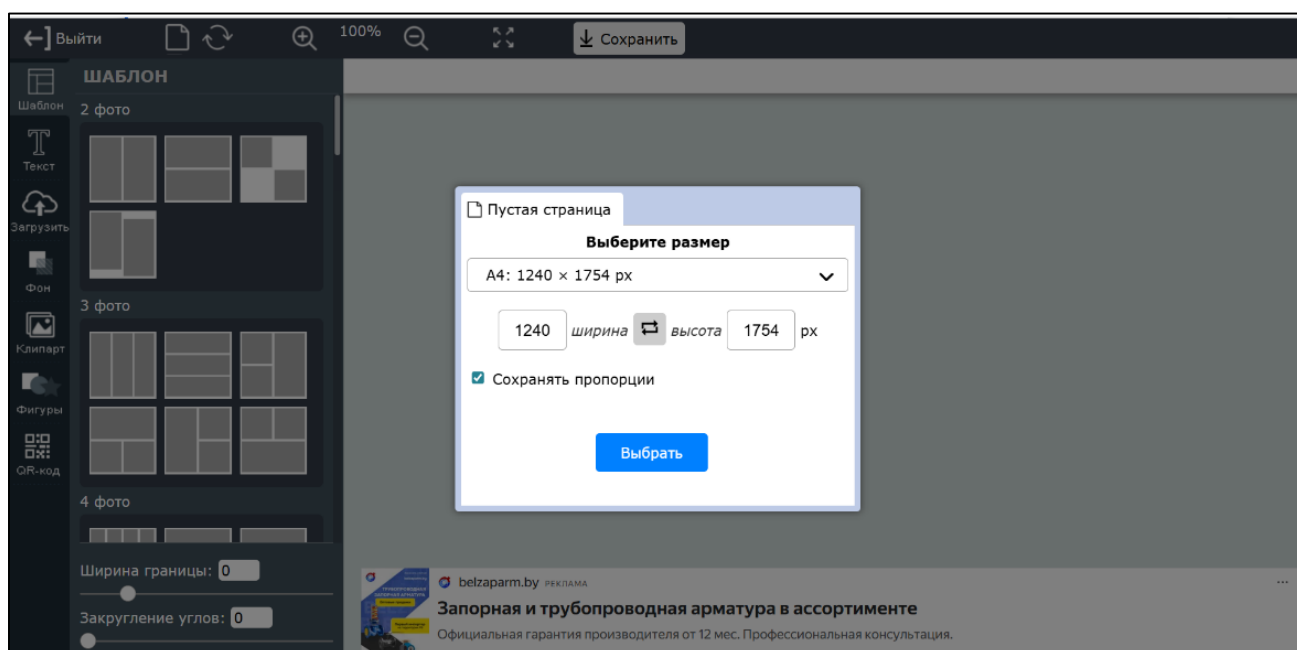


Рисунок 1.6 – Главная страница web-сайта «Progif»

Отличие ресурса от узких «сеточных» конструкторов в том, что наряду с быстрым объединением фотографий предлагается осмысленный набор макетов,

внутри которых уже задано взаимное расположение кадров; пользователь загружает изображения, подстраивает расположение в рамках выбранного шаблона, при необходимости усиливает выразительность снимка фильтрами и добавляет текстовые подписи. Дополнительно обычно доступны настройка размеров холста и сохранение результата в распространённых графических форматах, что закрывает типичный сценарий «сделал коллаж – получил файл для публикации или печати».

Требования к учётной записи и полнота функций без входа зависят от конкретной реализации сайта, однако для целевой аудитории важны понятный путь от главной страницы к редактору и отсутствие «порога вхождения» в духе профессиональных программ: интерфейс ориентирован на неспециалиста и предполагает последовательные шаги без перегруженности панелями.

Экспорт в привычных форматах поддерживается, но в бесплатной версии модель распространения сервиса часто проявляется в ограничениях: например, в водяном знаке на итоговом файле или в сокращённом наборе опций сохранения. Пример рабочей области редактора с выбранным макетом и элементами оформления приведён на рисунке 1.7.

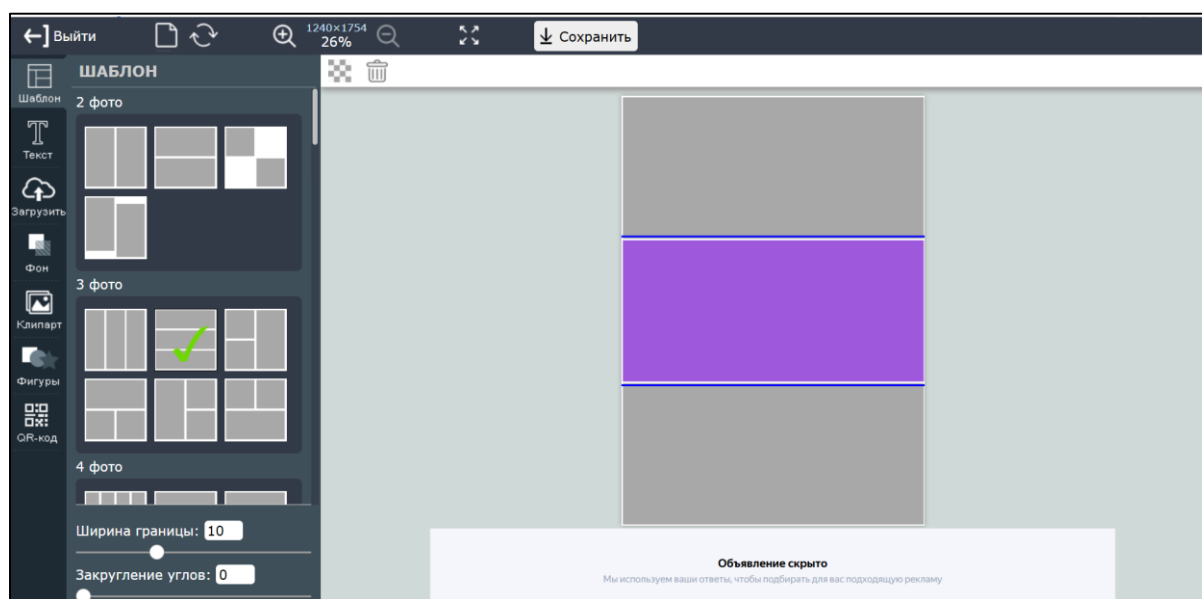


Рисунок 1.7 – Пример редактора

В совокупности к преимуществам относятся простота сценария, наличие бесплатных шаблонов и быстрая обработка изображений в рамках заложенных инструментов; к недостаткам – ограниченная кастомизация элементов по сравнению с универсальными редакторами, отсутствие продвинутой ретуши и зависимость качества бесплатного экспорта от ограничений, накладываемых сервисом на бесплатный тариф.

1.2 Постановка задачи

Web-приложение «Конструктор коллажей» должно обеспечивать создание, редактирование и хранение пользовательских графических проектов в виде колла-

жей. Сервис ориентирован на упрощение процесса работы с визуальным контентом за счёт предоставления удобного интерфейса и широкого набора инструментов.

Пользователи, не прошедшие аутентификацию (гости), могут зарегистрироваться и авторизоваться в системе. После входа в систему пользователи с ролью «Клиент» получают доступ к основному функционалу приложения.

Клиенты могут создавать новые коллажи, редактировать существующие проекты, выбирать размер холста, добавлять изображения, текстовые элементы и графические объекты (фигуры, стикеры, иконки). Также пользователям доступно применение фильтров к изображениям, использование встроенных шаблонов и создание собственных шаблонов.

Сервис должен поддерживать возможность отмены и повторения действий (Undo/Redo), использование сетки для выравнивания элементов, загрузку изображений на сервер и использование встроенной библиотеки изображений. Пользователь может просматривать галерею своих проектов, выполнять их сортировку и фильтрацию, удалять проекты и экспортировать готовые коллажи.

Предварительный просмотр готового вида коллажа перед сохранением файла на устройство может выполняться средствами интерфейса редактора на клиенте. Отдельное хранение на сервере полной истории изменений проекта по шагам редактирования в разрабатываемой версии приложения не предусмотрено; отмена и повтор действий на холсте реализуются на стороне клиента до сохранения актуального состояния проекта на сервере.

1.3 Выводы по разделу

1. Рассмотрены три аналога, выявлены их возможности и ограничения в части сценариев создания коллажей.

2. На основе анализа сформулирована направленность разрабатываемого «Конструктора коллажей»: приложение в браузере с полноценным холстом и инструментами редактирования, сохранение проектов и медиатеки на сервере, разграничение доступа для неаутентифицированного пользователя (Гость) и зарегистрированного клиента с расширенным функционалом после входа. Определён перечень функций постановки задачи.

3. Уточнено, что хранение на сервере отражает текущее состояние проекта, а детальная история правок на холсте обеспечивается на клиенте до сохранения.

Таким образом, в рамках курсового проекта необходимо разработать интернет-сервис «Конструктор коллажей» с клиентской частью на React и Vite, серверной частью на Node.js и Express, базой данных PostgreSQL и доступом к данным через ORM Prisma, развёртыванием в среде Docker Compose и публикацией пользователю через обратный прокси Nginx; обеспечить роли «Гость» и «Клиент», реализовать REST API и функциональность, согласованную с постановкой задачи и результатами анализа аналогов.

2 Проектирование web-приложения

2.1 Функциональные возможности web-приложения

На диаграмме вариантов использования (use case), представленной на рисунке 2.1, отображены ключевые функциональные возможности web-приложения и связи между актёрами и прецедентами, что облегчает понимание назначения разрабатываемой системы.

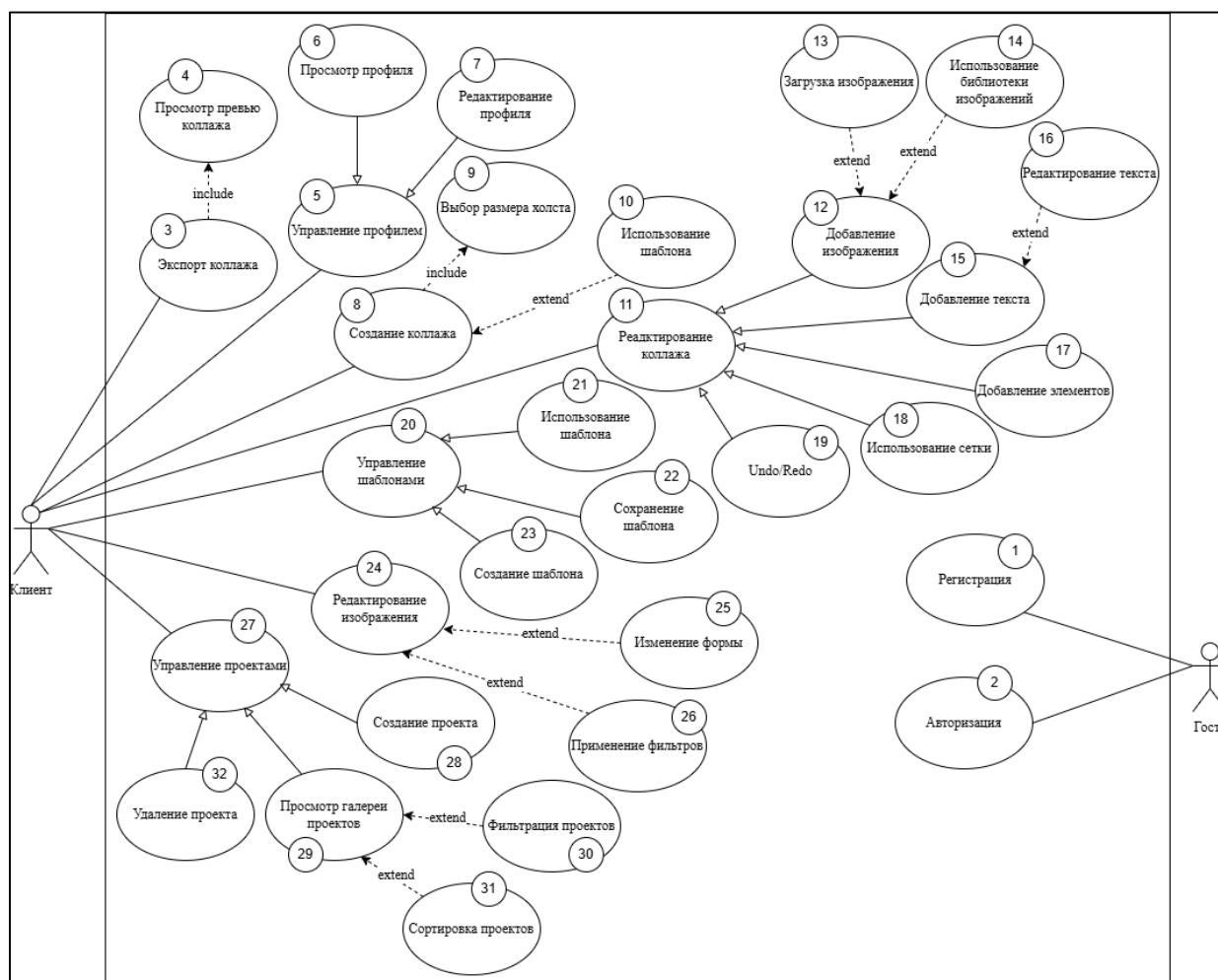


Рисунок 2.1 – Диаграмма вариантов использования web-приложения

На приведённой диаграмме вариантов использования отражён основной функционал web-приложения в разрезе двух ролей пользователей: гость и клиент.

На диаграмме актёр «Гость» связан с прецедентами регистрации и авторизации: регистрацию проходит неаутентифицированный пользователь, в результате в системе создаётся учётная запись с ролью USER, соответствующая роли «Клиент» в предметной модели. Прецеденты создания и редактирования коллажей, работы с проектами, шаблонами и медиатекой относятся к актёру «Клиент».

Краткое пояснение назначения каждой из этих ролей и их места в пользовательской модели системы содержится в таблице 2.1.

Таблица 2.1 – Назначение ролей пользователей в web-приложении

Роль	Назначение
Гость	Пользователь, не зарегистрированный в системе. Может проходить регистрацию и авторизацию для получения расширенного функционала
Клиент	Зарегистрированный пользователь системы, имеющий доступ ко всем функциям создания, редактирования, хранения и экспорта коллажей, а также управлению своим профилем и проектами

Функциональные возможности пользователя с ролью «Гость» представлены в таблице 2.2.

Таблица 2.2 – Функциональные возможности

Вариант использования	Пояснение
Регистрация	Процесс создания учетной записи в системе, включающий ввод необходимых данных для получения доступа к функционалу сервиса
Авторизация	Процесс входа в систему с использованием логина и пароля для получения доступа к функционалу интернет-сервиса

Функциональные возможности пользователя с ролью «Клиент» представлены в таблице 2.3.

Таблица 2.3 – Функциональные возможности

Вариант использования	Пояснение
1	2
Просмотр и редактирование профиля	Клиент может просматривать и изменять свои персональные данные
Создание нового коллажа	Пользователь может создавать новый проект коллажа с нуля
Редактирование существующего коллажа	Возможность изменения ранее созданных проектов
Экспорт коллажа	Сохранение готового коллажа в выбранном формате
Выбор размера холста	Настройка рабочей области под нужные параметры
Добавление и редактирование изображений	Загрузка изображений и их последующая обработка (масштабирование, перемещение, обрезка)
Добавление и редактирование текста	Вставка текстовых элементов с возможностью настройки шрифта, размера и стиля
Добавление элементов (фигуры, стикеры)	Использование графических элементов для оформления коллажа
Undo/Redo действия	Возможность отмены и возврата выполненных действий

Продолжение таблицы 2.3

1	2
Использование сетки	Использование сетки – включение на холсте вспомогательной сетки для точного выравнивания объектов
Использование встроенных шаблонов	Выбор готового макета из системного набора, чтобы быстрее начать коллаж с уже заданной композицией
Создание и сохранение собственных шаблонов	Пользователь может сохранять собственные макеты для повторного использования
Просмотр галереи проектов	Доступ к списку всех созданных пользователем коллажей
Удаление проектов	Удаление ранее созданных коллажей
Сортировка и фильтрация проектов	Упорядочивание и поиск проектов по названию, дате и другим доступным в интерфейсе критериям
Загрузка изображений на сервер	Добавление собственных изображений в облачное хранилище сервиса
Использование встроенной библиотеки изображений	Доступ к готовому набору изображений системы
Просмотр превью коллажа	Предварительный просмотр результата перед экспортом
Применение фильтров к изображениям	Наложение визуальных эффектов на выбранное изображение на холсте с сохранением результата в проекте

Таким образом, для каждой из введённых в предметную область ролей сформирован набор допустимых действий и связанных с ними возможностей системы: для гостя это сценарии, не требующие полного доступа к персональным данным и сохранённым на сервере проектам, а для клиента – полный цикл работы с коллажами и сопутствующими ресурсами после успешной аутентификации. Роль «Клиент» логически включает в себя весь функционал роли «Гость»: зарегистрированный пользователь сохраняет возможность выполнять базовые операции, доступные до входа (в частности, переход к регистрации и авторизации), и дополнительно получает расширенный доступ к защищённым функциям интернет-сервиса.

2.2 Проектирование базы данных

В рамках проектирования web-приложения была разработана база данных, предназначенная для надёжного хранения и согласованного управления всей информацией, без которой невозможно устойчивое функционирование интернет-сервиса создания и редактирования коллажей: учётные данные и атрибуты пользователей, проекты с состоянием холста, элементы композиции при необходимости нормализованного учёта, шаблоны и сведения о медиафайлах. Структура базы данных отражает выделенные сущности предметной области и связи между ними; её логическая схема приведена на рисунке 2.2.

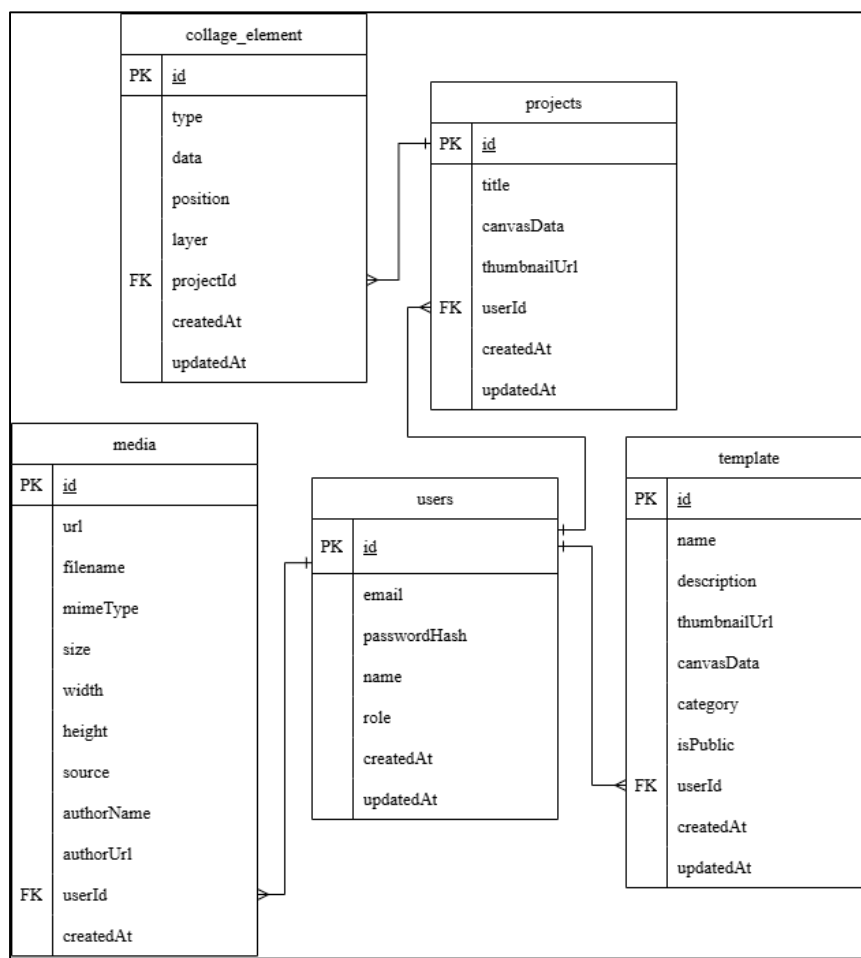


Рисунок 2.2 – Логическая схема базы данных web-приложения

База данных содержит пять основных таблиц, назначение каждой из которых описано в таблице 2.4.

Таблица 2.4 – Назначение таблиц базы данных

Таблица	Назначение
Users	Таблица, содержащая информацию о пользователях системы, включая учетные данные, имя и роль (гость, пользователь). Обеспечивает управление доступом к функционалу сервиса.
Projects	Таблица, содержащая информацию о созданных пользователями проектах коллажей, включая структуру холста, миниатюру и привязку к пользователю.
Collage_Elements	Таблица, хранящая отдельные элементы, их параметры, позицию и слой.
Templates	Таблица, содержащая шаблоны коллажей, доступные пользователям, включая публичные и пользовательские шаблоны.
Media	Таблица, предназначенная для хранения загруженных пользователями изображений и ресурсов, а также изображений из внешних источников.

В таблице Users хранится информация о пользователях web-приложения. Содержащиеся в ней поля описаны в таблице 2.5.

Таблица 2.5 – Структура таблицы Users

Название	Тип данных	Описание
id	uuid	Уникальный идентификатор пользователя
email	varchar	Электронная почта пользователя
passwordHash	varchar	Хэш пароля пользователя
name	varchar	Имя пользователя
role	varchar	Роль пользователя (GUEST, USER)
createdAt	timestamp	Дата создания учетной записи
updatedAt	timestamp	Дата последнего обновления

В таблице Projects хранится информация о проектах коллажей пользователей. Содержащиеся в ней поля описаны в таблице 2.6.

Таблица 2.6 – Структура таблицы Projects

Название	Тип данных	Описание
id	uuid	Уникальный идентификатор проекта
title	varchar	Название проекта
canvasData	json	Структура холста и содержимое коллажа
thumbnailUrl	varchar	Ссылка на превью проекта
userId	uuid	Идентификатор пользователя (внешний ключ)
createdAt	timestamp	Дата создания проекта
updatedAt	timestamp	Дата последнего изменения

В таблице Collage_Elements хранятся отдельные элементы коллажа. Содержащиеся в ней поля описаны в таблице 2.7.

Таблица 2.7 – Структура таблицы Collage_Elements

Название	Тип данных	Описание
id	uuid	Уникальный идентификатор элемента
type	varchar	Тип элемента (IMAGE, TEXT, SHAPE, STICKER)
data	json	Данные элемента
position	json	Координаты и размеры элемента на холсте
layer	int	Слой отображения элемента
projectId	uuid	Идентификатор проекта (внешний ключ)
createdAt	timestamp	Дата создания
updatedAt	timestamp	Дата обновления

В таблице Templates хранится информация о шаблонах коллажей. Содержащиеся в ней поля описаны в таблице 2.8.

Таблица 2.8 – Структура таблицы Templates

Название	Тип данных	Описание
id	uuid	Уникальный идентификатор шаблона
name	varchar	Название шаблона
description	varchar	Описание шаблона
thumbnailUrl	varchar	Превью шаблона
canvasData	json	Структура шаблона
category	varchar	Категория шаблона
isPublic	boolean	Признак публичности шаблона
userId	uuid	Идентификатор пользователя
createdAt	timestamp	Дата создания
updatedAt	timestamp	Дата обновления

В таблице Media хранится информация о загруженных изображениях и медиафайлах. Содержащиеся в ней поля описаны в таблице 2.9.

Таблица 2.9 – Структура таблицы Media

Название	Тип данных	Описание
id	uuid	Уникальный идентификатор медиафайла
url	varchar	Ссылка на файл
filename	varchar	Имя файла
mimeType	varchar	Тип файла
size	int	Размер файла
width	int	Ширина изображения
height	int	Высота изображения
source	varchar	Источник (UPLOAD, UNSPLASH)
authorName	varchar	Автор изображения
authorUrl	varchar	Ссылка на автора
userId	uuid	Идентификатор пользователя (внешний ключ)
createdAt	timestamp	Дата загрузки

Связи между таблицами базы данных интернет-сервиса представлены в таблице 2.10.

Таблица 2.10 – Назначение связей между таблицами

Таблица источник	Связанная таблица	Тип связи	Описание
1	2	3	4
Users	Projects	Один ко многим	Один пользователь может создавать несколько проектов
Users	Media	Один ко многим	Пользователь может загружать множество медиафайлов
Users	Templates	Один ко многим	Пользователь может создавать собственные шаблоны

Продолжение таблицы 2.10

1	2	3	4
Projects	Collage_Elements	Один ко многим	Один проект содержит множество элементов коллажа
Projects	Users	Многие к одному	Каждый проект принадлежит одному пользователю
Templates	Users	Многие к одному	Шаблон может принадлежать пользователю

Таким образом, каждая таблица базы данных имеет чётко определённое назначение и согласованную структуру полей и связей: совокупность таблиц покрывает учёт пользователей и их ролей, проекты коллажей с сохранённым состоянием холста и превью, нормализованное описание элементов композиции при необходимости отдельного хранения, каталог шаблонов с признаком публичности и авторством, а также метаданные загруженных и импортированных медиафайлов.

За счёт внешних ключей и каскадных правил обеспечивается целостность данных при удалении пользователя или проекта и предсказуемое поведение приложения при типовых операциях API.

2.3 Архитектура web-приложения

Клиент-серверная архитектура является основой приложения: пользовательский интерфейс отделён от логики доступа к данным и выполняется в браузере, а сервер принимает запросы, проверяет права и обращается к базе. Web-приложение состоит из двух основных компонентов – клиентской и серверной частей, обменивающихся данными по протоколу HTTP в формате, принятом для REST API.

В качестве сервера выступает приложение на платформе Node.js 20 с фреймворком Express 4.18.2, через который подключаются маршруты, промежуточные слои и обработчики ошибок.

Для работы с базой данных используется ORM Prisma 5.10.2: она задаёт модель данных декларативно, обеспечивает типобезопасный доступ к записям и поддерживает эволюцию схемы (в том числе миграции в процессе разработки). В качестве СУБД выбрана PostgreSQL 15. Аутентификация пользователей реализуется на основе JWT с библиотекой jsonwebtoken 9.0.2 [14], пароли перед сохранением хешируются с помощью bcryptjs 2.4.3 [15], а загрузка файлов на сервер выполняется посредством Multer 2.1.1 [16]. Клиентская часть разрабатывается на React.js 18.2.0 с HTTP-клиентом Axios 1.6.7 [17] и навигацией между страницами SPA на React Router DOM 6.22.1 [18].

Серверные контейнеры в составе Docker собираются на лёгких образах Linux Alpine: node:20-alpine, postgres:15-alpine, nginx:alpine. Клиент на React подготавливается и отлаживается с использованием сборщика Vite версии 5.1.4; запуск связки сервисов описывается и автоматизируется через Docker Compose. Для подбора стоковых изображений используется внешний сервис Unsplash API [19], вызываемый с серверной стороны по ключу доступа.

Схема развертывания данного приложения представлена на рисунке 2.3.

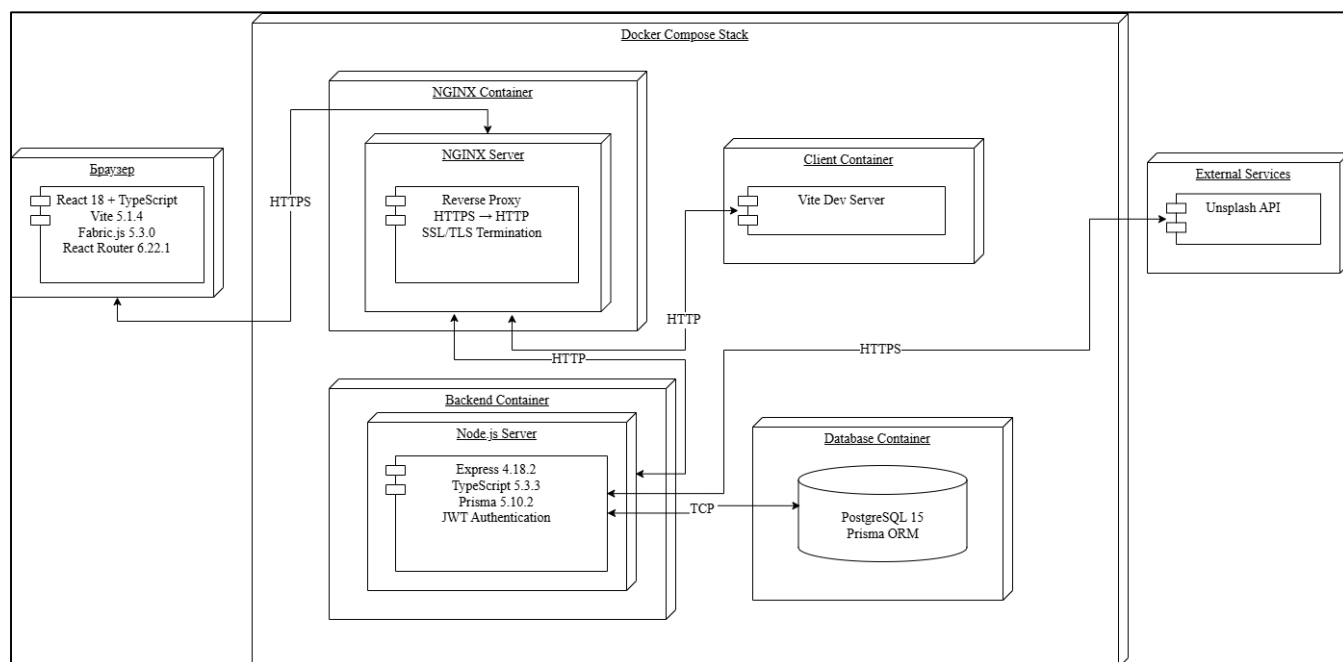


Рисунок 2.3 – Схема развёртывания

Инфраструктура включает четыре сервиса: сервер базы данных PostgreSQL, бэкенд-сервер Node.js, фронтенд-сервер Vite и реверс-прокси Nginx.

Для обеспечения безопасности передачи данных между клиентом и сервером используется протокол HTTPS, реализованный посредством SSL/TLS-сертификатов, подключённых к Nginx. Все входящие HTTP-запросы на порт 80 автоматически перенаправляются на HTTPS-порт 443, что исключает передачу данных в незашифрованном виде. Поддерживаются протоколы TLSv1.2 и TLSv1.3. Протокол HTTP 1.1 будет использоваться для передачи данных между клиентом и сервером. Протокол TCP будет применяться для связи между бэкенд-сервером и сервером базы данных, обеспечивая надежную передачу данных. Для поиска и добавления стоковых изображений в коллаж будет использоваться сервис Unsplash API, который является внешним облачным сервисом хранения и предоставления фотографий. Загруженные пользователем файлы хранятся локально на сервере в директории /uploads и раздаются через Express в качестве статических файлов.

2.4 Выводы по разделу

1. Определены актёры и прецеденты; зафиксировано соответствие роли «Клиент» учётной записи с ролью USER в базе данных.
2. Спроектирована база данных из пяти таблиц; описаны поля и связи между сущностями.
3. Выбран стек технологий с указанием версий ключевых компонентов; описаны протоколы HTTPS, HTTP/1.1 и TCP; отражено использование Docker Compose и Nginx.
4. Задана схема развёртывания с разделением клиента, API, СУБД и обратного прокси.

3 Реализация web-приложения

3.1 Программная платформа Node.js

Для серверной части проекта выбрана среда Node.js 20 – платформа выполнения JavaScript на стороне сервера с неблокирующей моделью ввода-вывода, удобной для построения HTTP API и работы с базами данных.

В рамках проекта сервер реализован на TypeScript. Обмен данными с клиентом выполняется в формате JSON; для загрузки файлов применяется Multer, для аутентификации – JSON Web Token и хеширование паролей. Кросс-доменные запросы обрабатываются middleware CORS [20].

В качестве HTTP-фреймворка используется Express.js, обеспечивающий маршрутизацию, подключение middleware, раздачу статических файлов и централизованную обработку ошибок. Полный текст точки входа сервера и объявления подключения маршрутов приведён в приложении А; на листинге 3.1 ниже показан фрагмент для иллюстрации структуры API.

```
const app = express();
const prisma = new PrismaClient();
const PORT = process.env.PORT || 3000;
app.use(cors());
app.use(express.json({ limit: '50mb' }));
app.use(express.urlencoded({ limit: '50mb', extended: true }));
app.use('/uploads', express.static(path.join(process.cwd(), 'uploads')));
import authRoutes from './routes/auth.routes';
import userRoutes from './routes/user.routes';
import projectRoutes from './routes/project.routes';
import templateRoutes from './routes/template.routes';
import imageRoutes from './routes/image.routes';
app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);
app.use('/api/projects', projectRoutes);
app.use('/api/templates', templateRoutes);
app.use('/api/images', imageRoutes);
app.get('/api/health', async (req, res, next) => {
  try {await prisma.$queryRaw`SELECT 1`;
    res.status(200).json({ status: 'ok', db: 'connected', version: '1.0' });} catch (error) {next(error);}});
app.use((err: Error, req: Request, res: Response, next: NextFunction) => {
  res.status(500).json({ error: 'Internal Server Error', message: err.message });});
app.listen(PORT, ()=>{console.log(`Server on port ${PORT}`);});
```

Листинг 3.1 – Фрагмент настройки сервера Express и API-маршрутов

Таким образом, серверная часть конструктора коллажей опирается на связку Node.js и Express как на основу транспортного уровня REST API, а доступ к данным инкапсулируется через клиент Prisma.

3.2 Система управления базами данных PostgreSQL

Для хранения учётных записей, проектов коллажей, шаблонов и метаданных медиафайлов используется PostgreSQL 15 – реляционная СУБД с поддержкой транзакций ACID, ограничений целостности и расширенного набора типов данных. Для проекта важны в первую очередь тип JSON/, связи «один ко многим» и каскадное удаление зависимых записей при удалении пользователя или проекта.

Развёртывание СУБД в среде разработки выполняется контейнером postgres:15-alpine, что обеспечивает воспроизводимость окружения и простое резервирование данных томом Docker.

3.3 Реализация базы данных

Для объектно-реляционного доступа к PostgreSQL применяется ORM Prisma: схема базы задаётся декларативно в файле schema.prisma, после чего генерируется типобезопасный клиент @prisma/client.

В schema.prisma также заданы перечисления: Role (роли пользователя в системе), MediaSource (происхождение медиафайла), ElementType (вид элемента на холсте коллажа). Такое представление ограничивает допустимые значения на уровне модели и упрощает проверку данных при сохранении проектов и медиатеки. Перечисления и блок подключения к базе данных приведены в листинге 3.2.

```
generator client {
  provider      = "prisma-client-js"
  binaryTargets = ["native", "linux-musl", "linux-musl-openssl-3.0.x"]}
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")}
enum Role {
  GUEST
  USER
  ADMIN}
enum MediaSource {
  UPLOAD
  UNSPLASH}
enum ElementType {
  IMAGE
  TEXT
  SHAPE
  STICKER}
```

Листинг 3.2 – Подключение к PostgreSQL и перечисления

Модель User описывает пользователя сервиса: уникальный идентификатор, электронную почту, хеш пароля, имя, роль и временные метки. С пользователем связаны коллекции проектов, шаблонов и загруженных медиафайлов; при удалении пользователя связанные записи удаляются каскадно согласно настройкам отношений в Prisma. Модель User приведена в листинге 3.3.

```

model User {
  id          String      @id @default(uuid())
  email       String?     @unique
  passwordHash String?
  name        String?
  role        Role        @default(GUEST)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  projects    Project[]
  templates   Template[]
  media       Media[]
  @@map("users")
}

```

Листинг 3.3 – Модель пользователя

Модель Project соответствует сохранённому коллажу: название, принадлежность пользователю, JSON-поле canvasData с полным состоянием холста (объекты Fabric.js, размеры), опциональная ссылка на превью (миниатюра экспорта). Отдельная связь с CollageElement позволяет при необходимости дублировать или детализировать элементы на уровне строк таблицы. Модель Project приведена в листинге 3.4.

```

model Project
{
  id          String      @id @default(uuid())
  title       String
  canvasData  Json
  thumbnailUrl String?
  userId      String

  user        User        @relation(fields: [userId], references:
[id], onDelete: Cascade)

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  elements    CollageElement[]
  @@map("projects")
}

```

Листинг 3.4 – Модель проекта

Модель CollageElement предназначена для хранения отдельного элемента коллажа. В ней фиксируется тип элемента (изображение, текст, фигура, стикер), произвольные атрибуты в формате JSON, положение и геометрия на холсте, порядок наложения (слой), а также внешний ключ на родительский проект. Такое представление сочетает нормализованное хранение с гибкими данными под каждый тип объекта и обеспечивает согласованность между состоянием на сервере и логикой редактирования на клиенте. Структура модели элемента коллажа приведена в листинге 3.5.

```

model CollageElement {

    id          String          @id @default(uuid())
    type        ElementType
    data        Json
    position    Json
    layer       Int
    projectId   String

    project     Project          @relation(fields: [projectId], references:
[id], onDelete: Cascade)

    createdAt   DateTime         @default(now())
    updatedAt   DateTime         @updatedAt

    @@map("collage_elements")
}

```

Листинг 3.5 – Модель элемента коллажа

Модель Template описывает шаблон оформления: имя, описание, превью, данные холста, категорию, признак публичности и опциональную связь с автором. Модель шаблона представлена в листинге 3.6.

```

model Template {

    id          String          @id @default(uuid())
    name        String
    description  String?
    thumbnailUrl String?
    canvasData  Json
    category     String         @default("Other")
    isPublic     Boolean        @default(false)
    userId       String?

    user         User?          @relation(fields: [userId], references:
[id], onDelete: Cascade)

    createdAt    DateTime       @default(now())
    updatedAt    DateTime       @updatedAt

    @@map("templates")
}

```

Листинг 3.6 – Модель шаблона

Модель Media фиксирует загруженные или импортированные изображения: URL, имя файла, MIME-тип, размер, габариты, источник, атрибуцию автора при работе с внешним API. Модель медиафайла представлена в листинге 3.7.

```

model Media {
  id          String          @id @default(uuid())
  url         String
  filename    String?
  mimeType    String?
  size        Int?
  width       Int?
  height      Int?
  source      MediaSource @default(UPLOAD)
  authorName  String?
  authorUrl   String?
  userId      String
  user        User           @relation(fields: [userId], references:
[id], onDelete: Cascade)
  createdAt   DateTime       @default(now())
  @@map("media")
}

```

Листинг 3.7 – Модель медиафайла

Полный декларативный текст файла `prisma/schema.prisma` с описанием всех моделей и перечислений приведён в приложении Б.

В результате проектирования получена согласованная схема из пяти основных сущностей и три перечисления; основные связи: пользователь – проекты и медиа (один ко многим), проект – элементы коллажа (один ко многим), пользователь – шаблоны (один ко многим, с допуском системных шаблонов без `userId`). Экземпляр `PrismaClient` создаётся при старте сервера и используется в контроллерах для выполнения запросов.

3.4 Программные библиотеки

В процессе разработки серверной части web-приложения использовались библиотеки, перечисленные в таблице 3.1. Прямое подключение драйвера `pg` не требуется: доступ к PostgreSQL осуществляется через `Prisma Client`, который использует собственный механизм подключения. Хеширование паролей выполняется пакетом `bcryptjs`. Для типизации запросов с загрузкой файлов в зависимости указан пакет `@types/multer`.

Таблица 3.1 – Программные библиотеки серверной части

Библиотека	Версия	Назначение
1	2	3
express	4.18.2	HTTP-фреймворк: маршруты, middleware, обработка запросов и ответов
bcryptjs	2.4.3	Хеширование паролей перед сохранением в базе данных
cors	2.8.5	Включение политики CORS для запросов из браузера к API
dotenv	16.4.5	Загрузка параметров окружения из файла <code>.env</code>

Продолжение таблицы 3.1

1	2	3
jsonwebtoken	9.0.2	Формирование и проверка JWT при аутентификации
multer	2.1.1	Приём multipart/form-data при загрузке изображений на сервер
@prisma/client	5.10.2	Клиент ORM для типобезопасных запросов к PostgreSQL
prisma	5.10.2	CLI: генерация клиента, prisma db push в режиме разработки

Для клиентской части задействованы библиотеки по таблице 3.2. Интерфейс построен на React: состояние авторизации локализовано в контексте приложения. Сборка и локальный dev-сервер выполняются Vite с плагином @vitejs/plugin-react. Редактор коллажа использует Fabric.js для работы с элементами на HTML Canvas.

Таблица 3.2 – Программные библиотеки клиентской части

Библиотека	Версия	Назначение
react	18.2.0	Библиотека для построения пользовательского интерфейса из компонентов
react-dom	18.2.0	Рендеринг React в DOM браузера
react-router-dom	6.22.1	Маршрутизация страниц SPA
axios	1.6.7	HTTP-клиент для вызовов REST API
fabric	5.3.0	Canvas-движок: объекты холста, слои, экспорт изображения
vite	5.1.4	Сборка и dev-сервер для клиентского приложения
@vitejs/plugin-react	4.2.1	Интеграция React в сборку Vite
typescript	5.2.2	Типизация компонентов и сервисов на клиенте

Перечисленный набор покрывает маршрутизацию и формы на клиенте, работу с API и холстом, а на сервере – безопасность, загрузку файлов и персистентность данных через ORM.

3.5 Паттерны

На сервере реализован стиль REST API: маршруты объявляются в модулях routes, обработка запросов вынесена в контроллеры controllers, проверка JWT и приём файлов оформлены как middleware. На клиенте состояние авторизации передаётся через контекст React, страницы построены как композиция компонентов.

3.6 Описание серверной части

Серверная часть реализована на Node.js и организована по принципу разделения маршрутов, контроллеров и middleware. Загрузки сохраняются в каталог uploads относительно рабочей директории сервера и отдаются как статические файлы по URL-префиксу /uploads. Структура каталогов приведена в таблице 3.3.

Таблица 3.3 – Директории серверной части проекта

Директория	Назначение
prisma/	Файл <code>schema.prisma</code> – модель данных и подключение к PostgreSQL
src/	Исходный код сервера на TypeScript.
src/controllers/	Аутентификация, пользователи, проекты, шаблоны, изображения.
src/middlewares/	Проверка JWT, настройка Multer для загрузки файлов.
src/routes/	Объявление маршрутов API и привязка к контроллерам.
src/index.ts	Точка входа: создание приложения Express, подключение middleware и маршрутов, эндпоинт проверки состояния.
src/seed.ts	Скрипт начального заполнения
uploads/	Файлы, загруженные пользователями

Сопоставление HTTP-методов и путей с контроллерами и функциями приведено в таблице 3.4. Префикс `/api` задаётся при монтировании маршрутов в `index.ts`.

Таблица 3.4 – Контроллеры и функции маршрутов

Метод	Маршрут (относительно <code>/api</code>)	Файл контроллера	Функция
GET	<code>/health</code>	<code>index.ts</code>	Проверка доступности API и подключения к БД
POST	<code>/auth/register</code>	<code>auth.controller.ts</code>	<code>register</code>
POST	<code>/auth/login</code>	<code>auth.controller.ts</code>	<code>login</code>
GET	<code>/auth/me</code>	<code>auth.controller.ts</code>	<code>getMe</code>
PUT	<code>/users/profile</code>	<code>user.controller.ts</code>	<code>updateProfile</code>
GET	<code>/projects/</code>	<code>project.controller.ts</code>	<code>getProjects</code>
POST	<code>/projects/</code>	<code>project.controller.ts</code>	<code>createProject</code>
GET	<code>/projects/:id</code>	<code>project.controller.ts</code>	<code>getProjectById</code>
PUT	<code>/projects/:id</code>	<code>project.controller.ts</code>	<code>updateProject</code>
DELETE	<code>/projects/:id</code>	<code>project.controller.ts</code>	<code>deleteProject</code>
GET	<code>/templates/</code>	<code>template.controller.ts</code>	<code>getTemplates</code>
GET	<code>/templates/:id</code>	<code>template.controller.ts</code>	<code>getTemplateById</code>
POST	<code>/templates/</code>	<code>template.controller.ts</code>	<code>createTemplate</code>
DELETE	<code>/templates/:id</code>	<code>template.controller.ts</code>	<code>deleteTemplate</code>
GET	<code>/images/unsplash</code>	<code>image.controller.ts</code>	<code>searchUnsplash</code>
POST	<code>/images/unsplash/track</code>	<code>image.controller.ts</code>	<code>trackUnsplashDownload</code>
GET	<code>/images/library</code>	<code>image.controller.ts</code>	<code>getLibrary</code>
POST	<code>/images/upload</code>	<code>image.controller.ts</code>	<code>uploadImage</code>
DELETE	<code>/images/:id</code>	<code>image.controller.ts</code>	<code>deleteMedia</code>

Обмен данными между клиентом и сервером в сценариях выполняется в формате JSON. Тело ответа при регистрации и входе включает JWT, который клиент сохраняет и передаёт в заголовке `Authorization`.

3.7 Реализация функций пользователя с ролью «Гость»

Гость не имеет действующего сеанса и не может вызывать защищённые маршруты, включая список проектов, редакторские операции с проектами на сервере, шаблоны и медиатеку. Доступны регистрация и вход.

3.7.1 Аутентификация

Гость может войти в систему, отправив email и пароль на маршрут POST /api/auth/login. Контроллер ищет пользователя в таблице users, сравнивает пароль с хешем через bcrypt.compare и при успехе возвращает данные пользователя и JWT. Ниже приведена реализация функции login. Полная реализация функций входа и вспомогательных проверок учётных данных приведена в приложении В. Фрагмент функции login представлен в листинге 3.8.

```
export const login = async (req: Request, res: Response) => {
  try {
    const { email, password } = req.body;
    const user = await prisma.user.findUnique({ where: { email } });
    if (!user || !user.passwordHash) {
      return res.status(401).json({ message: 'Неверный email или пароль' });
    }
    const isMatch = await bcrypt.compare(password, user.passwordHash);
    if (!isMatch) {
      return res.status(401).json({ message: 'Неверный email или пароль' });
    }
    res.json({
      id: user.id,
      name: user.name,
      email: user.email,
      role: user.role,
      token: generateToken(user.id),
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Ошибка сервера при входе' });
  }
};
```

Листинг 3.8 – Реализация функции login

Функция извлекает учётные данные из тела запроса, выполняет проверку существования записи и совпадения пароля, после чего формирует ответ с токеном, необходимым для последующих запросов от имени пользователя с ролью USER.

3.7.2 Регистрация

Регистрация нового пользователя реализована функцией `register` на маршруте `POST /api/auth/register`. Выполняется проверка обязательных полей, формата email и минимальной длины пароля, проверка уникальности email, хеширование пароля и создание записи с ролью `USER`. При успехе клиент получает идентификатор, профильные поля и JWT. Реализация функции `register` представлена в листинге 3.9.

```
export const register = async (req: Request, res: Response) => {
  try {
    const { name, email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ message: 'Email и пароль обязательны' });
    }
    if (password.length < 6) {
      return res.status(400).json({ message: 'Пароль должен быть не менее 6 символов' });
    }
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email)) {
      return res.status(400).json({ message: 'Некорректный формат email' });
    }
    const userExists = await prisma.user.findUnique({ where: { email } });
    if (userExists) {
      return res.status(400).json({ message: 'Пользователь с таким email уже существует' });
    }
    const salt = await bcrypt.genSalt(10);
    const passwordHash = await bcrypt.hash(password, salt);
    const user = await prisma.user.create({
      data: {
        name,
        email,
        passwordHash,
        role: 'USER',
      },
    });
    res.status(201).json({
      id: user.id,
      name: user.name,
      email: user.email,
      role: user.role,
      token: generateToken(user.id),
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Ошибка сервера при регистрации' });
  }
};
```

Листинг 3.9 – Реализация функции `register` (`auth.controller.ts`)

Таким образом, гость после успешной регистрации получает токен и может работать с приложением как авторизованный пользователь; до регистрации операции с проектами на сервере остаются недоступны.

3.7.3 Проверка доступности сервиса

Для гостя и для мониторинга инфраструктуры предусмотрен публичный эндпоинт GET /api/health. Использование данного эндпоинта представлено в листинге 3.10.

```
app.get('/api/health', async (req, res, next) => {
  try {
    await prisma.$queryRaw`SELECT 1`;
    res.status(200).json({ status: 'ok', db: 'connected', version:
'1.0'
  });
  } catch (error) {
    console.error('Database connection failed', error);
    next(error);
  }
});
```

Листинг 3.10 – Проверка состояния API и БД (index.ts)

Использование данного маршрута удобно при развёртывании в Docker и при отладке клиента: до ввода учётных данных можно проверить связность «клиент – Nginx – сервер – PostgreSQL». Основной сценарий для гостя в приложении остаётся переход на страницы входа и регистрации.

3.8 Реализация функций пользователя с ролью «Клиент»

Под «клиентом» понимается аутентифицированный пользователь с ролью USER в базе данных. После входа клиент передаёт JWT в заголовке Authorization: Bearer <token>; middleware protect проверяет токен и помещает идентификатор пользователя в объект запроса.

В отличие от гостя, клиент может получать список своих проектов, создавать и редактировать коллажи на сервере, сохранять canvasData и миниатюру, удалять проекты, изменять имя в профиле, а также пользоваться API шаблонов и изображений.

3.8.1 Создание проекта коллажа

Клиент может создать новый проект запросом POST /api/projects. В теле передаются необязательные поля title, width, height; при их отсутствии подставляются значения по умолчанию. В базе создаётся запись с пустым массивом объектов холста в canvasData.

Функция создания проекта вместе с остальными операциями над проектами приведена в приложении Г. Реализация приведена в листинге 3.11.

```

export const createProject = async (req: AuthRequest, res: Response)
=> {
  try {
    const { title, width, height } = req.body;
    const userId = req.user?.id;
    if (!userId) return res.status(401).json({ message: 'Unauthorized' });
    const newProject = await prisma.project.create({
      data: {
        title: title || 'Новый проект',
        userId,
        canvasData: { width: width || 1080, height: height || 1080,
objects: [] },
      },
    });
    res.status(201).json(newProject);
  } catch (error) {
    console.error('Error creating project:', error);
    res.status(500).json({ message: 'Internal server error' });
  }
};

```

Листинг 3.11 – Функция createProject (project.controller.ts)

Метод извлекает идентификатор владельца из req.user, созданного после успешной проверки JWT, формирует начальное состояние холста и возвращает созданный проект со статусом 201.

3.8.2 Сохранение изменений проекта (обновление коллажа)

При работе в редакторе клиент отправляет PUT /api/projects/:id с обновляемыми полями: название, canvasData, thumbnailUrl для превью. Перед обновлением выполняется проверка: проект существует и принадлежит текущему пользователю; иначе возвращаются коды 404 или 403. Такая обработка ошибок представлена в функции updateProject. Обновление проекта и проверка принадлежности записи пользователю выполняются в коде, помещённом в приложение Г.

3.8.3 Удаление проекта

Удаление выполняется запросом DELETE /api/projects/:id. Перед удалением проверяется, что проект существует и принадлежит текущему пользователю; иначе возвращаются коды 404 или 403. При успешном удалении запись удаляется из таблицы projects; связанные записи при настроенном в схеме Prisma каскадном удалении удаляются автоматически. Реализация функции deleteProject представлена в Приложении Г.

3.8.4 Редактирование профиля

Изменение данных профиля реализовано запросом PUT /api/users/profile (маршруты пользователя монтируются с префиксом /api/users). В теле запроса передаётся новое отображаемое поле name; идентификатор пользователя берётся из req.user.id после проверки JWT в middleware, без передачи чужого идентификатора

в URL – это снижает риск несанкционированного изменения чужих записей. Исходный код функции `updateProfile` приведён в листинге 3.12.

```
export const updateProfile = async (req: AuthRequest, res: Response)
=> { try {
  const { name } = req.body;
  const updatedUser = await prisma.user.update({
    where: { id: req.user?.id },
    data: { name },
    select: { id: true, name: true, email: true, role: true, up-
datedAt: true });
  res.json(updatedUser);
} catch (error) {
  console.error(error);
  res.status(500).json({ message: 'Ошибка обновления профиля' });
}
}
```

Листинг 3.12 – Функция `updateProfile` (`user.controller.ts`)

Итог выполнения возвращается в формате JSON и может отображаться на странице профиля SPA. При необходимости расширения (смена email, удаление аккаунта) потребуется дополнить контроллер и политику валидации отдельными методами.

3.8.5 Получение списка шаблонов

Запрос GET `/api/templates` поддерживает фильтрацию query-параметрами `category` и `score`: собственные шаблоны пользователя (`my`), только публичные (`public`) или совмещённый режим (`all`). Результат упорядочивается по дате создания и может использоваться панелью шаблонов в редакторе. Работа со списком шаблонов и связанные операции шаблонов приведены в приложении Д.

3.8.6 Создание пользовательского шаблона

Сохранение текущего макета как шаблона выполняется POST `/api/templates`. Обязательны `name` и `canvasData`; запись связывается с `userId` текущего пользователя. Создание пользовательского шаблона содержится в приложении Д.

Ответ со статусом 201 содержит созданную сущность и может использоваться для обновления локального списка шаблонов в интерфейсе.

3.8.7 Удаление пользовательского шаблона

Удаление выполняется DELETE `/api/templates/:id`. Допускается удаление только шаблонов, у которых `userId` совпадает с текущим пользователем; системные шаблоны без владельца таким API не снимаются. Фрагмент логики удаления шаблона показан в листинге 3.13.

```
export const deleteTemplate = async (req: AuthRequest, res: Response) => { try {
  const { id } = req.params;
  const userId = req.user?.id;
  if (!userId) return res.status(401).json({ message: 'Unauthorized' });
  const template = await prisma.template.findUnique({ where: { id } });
  if (!template) return res.status(404).json({ message: 'Not found' });
  if (template.userId !== userId) return res.status(403).json({ message: 'Forbidden' });
  await prisma.template.delete({ where: { id } });
  res.json({ message: 'Deleted' }); } catch (error) {
  res.status(500).json({ message: 'Internal server error' });}};
```

Листинг 3.13 – Функция deleteTemplate (template.controller.ts)

После успешного удаления клиент исключает шаблон из пользовательского каталога.

3.8.8 Загрузка файла на сервер и пополнение медиатеки

Поиск изображений через Unsplash выполняется отдельными методами searchUnsplash и trackUnsplashDownload. Локальная медиатека пользователя пополняется загрузкой файла POST /api/images/upload с телом multipart/form-data и полем file. Multer сохраняет файл в каталог uploads; в базе создаётся запись Media. Реализация функции uploadImage приведена в листинге 3.14.

```
export const uploadImage = async (req: AuthRequest, res: Response) => { try {
  const userId = req.user?.id;
  if (!userId) return res.status(401).json({ message: 'Unauthorized' });
  const file = req.file;
  if (!file) return res.status(400).json({ message: 'Файл не найден' });
  const fileUrl = `/uploads/${file.filename}`;
  const media = await prisma.media.create({
    data: {
      url: fileUrl,
      source: 'UPLOAD',
      userId, } as any, });
  res.status(201).json(media); } catch (error) {
  console.error('uploadImage error:', error);
  res.status(500).json({ message: 'Internal server error' });}};
```

Листинг 3.14 – Функция uploadImage

Полученный url клиент подставляет в объекты изображений на холсте; список ранее загруженных файлов доступен через GET /api/images/library. Удаление

записи медиатеки и файла с диска реализовано в deleteMedia и дополняет жизненный цикл пользовательских ресурсов без обращения к отдельной «административной» консоли.

3.9 Структура клиентской части

Клиентская часть построена по компонентной модели React и собирается с помощью Vite. Глобальное состояние авторизации реализовано через контекст (AuthContext). Основные модули сосредоточены в каталоге client/src; назначение подкаталогов приведено в таблице 3.5.

Таблица 3.5 – Основные директории в папке src клиента и их назначение

Директория	Назначение
pages/	Страницы приложения: вход, регистрация, панель проектов, редактор коллажа, профиль.
components/	Переиспользуемые компоненты интерфейса: карточка проекта, модальные окна, обёртка маршрутов; вложенная папка editor/ – панели инструментов, холст, тулбар.
context/	React Context.
hooks/	Пользовательские хуки: работа с Fabric.js-холстом (useCanvas), горячие клавиши (useHotkeys).
services/	Обращения к REST API.
styles/	Глобальные стили и тема.

Маршрутизация задаётся в App.tsx через react-router-dom. Для роли «Гость» (неавторизованный пользователь) доступны страницы входа и регистрации. Для «Клиента» – страницы, обёрнутые в ProtectedRoute, который при отсутствии токена перенаправляет на /login.

Маршруты и компоненты страниц для пользователя с ролью «Гость» представлены в таблице 3.6.

Таблица 3.6 – Маршруты и компоненты страниц для роли «Гость»

Компонент страницы	Маршрут	Назначение
Login	/login	Форма входа, получение JWT
Register	/register	Форма регистрации нового пользователя

Маршрут /editor/:id? в текущей конфигурации не обёрнут в ProtectedRoute, поэтому страницу редактора теоретически можно открыть без входа, однако загрузка и сохранение проектов через API потребуют авторизации; на практике основной сценарий предполагает вход и переход из панели проектов. Маршруты и компоненты страниц для пользователя с ролью «Клиент» представлены в таблице 3.7.

Таблица 3.7 – Маршруты и компоненты страниц для роли «Клиент»

Компонент страницы	Маршрут	Назначение
Dashboard	/dashboard	Список коллажей пользователя, создание проекта, переход в редактор
Profile	/profile	Просмотр и изменение имени профиля
Editor	/editor/:id?	Редактор холста: инструменты, слои, шаблоны, экспорт

Вспомогательные и повторно используемые элементы интерфейса сведены в таблицу 3.8.

Таблица 3.8 – Вспомогательные и общие компоненты клиентской части

Компонент	Назначение
ProtectedRoute	Ограничивает доступ к вложенным страницам только для авторизованных пользователей
AuthProvider	Оборачивает приложение и предоставляет контекст авторизации дочерним компонентам
ProjectCard, ProjectModal	Отображение проекта в списке и модальное окно создания проекта
CanvasArea, CanvasWrapper	Область редактирования и связка с Fabric.js
Toolbar, ContextPanel	Панель инструментов и контекстные настройки объекта
Панели editor/panels/	Фон, шаблоны, стикеры, фигуры, свойства, слои, фильтры, рамки, обрезка, экспорт, история действий
Модальные окна editor/modals/	Предпросмотр, экспорт, поиск изображений (Unsplash), шаблоны, загрузка файлов
Toast	Всплывающие уведомления для пользователя
CropOverlay	Наложение для кадрирования изображения на холсте

Обработчик маршрута в App.tsx выполняет перенаправление на /dashboard. Для неавторизованного пользователя попытка открыть защищённую панель приведёт к редиректу на /login согласно логике ProtectedRoute.

3.10 Настройка конфигурации Docker

Развёртывание сервисов автоматизировано с помощью Docker и Docker Compose: это обеспечивает воспроизводимое окружение с PostgreSQL, Node.js-сервером API, Vite-клиентом и Nginx в качестве обратного прокси. В репозитории для клиента и сервера используются образы на базе node:20-alpine и файлы Dockerfile.dev.

Состав сервисов и их зависимости задаются в docker-compose.yml в корне проекта: контейнер сервера ожидает готовности базы данных по healthcheck, том uploads_data хранит загруженные файлы, том postgres_data – данные СУБД.

Образ клиента в режиме разработки собирается по client/Dockerfile.dev: устанавливаются зависимости npm install, затем по умолчанию запускается npm run dev для горячей перезагрузки. Пример Dockerfile клиента представлен в листинге 3.15.

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
CMD ["npm", "run", "dev"]
```

Листинг 3.15 – client/Dockerfile.dev

Полные конфигурационные файлы контейнерной сборки и обратного прокси без сокращений приведены в приложении Е.

Для управления контейнерами в каталоге с docker-compose.yml обычно используют команды: docker compose up -d, docker compose build, docker compose stop. В режиме Compose API доступно на хосте на порту 3000, клиент Vite – на 5173, PostgreSQL – на 5432.

3.11 Настройка конфигурации Nginx

В проекте используется официальный образ nginx:alpine. Файл nginx/default.conf задаёт проксирование /api/ и /uploads/ на сервис server:3000, а корень / – на dev-сервер client:5173 с поддержкой WebSocket для Vite. Содержимое файла nginx представлено в Приложении Е.

Использование Nginx позволяет сопоставить единую точку входа пользователя с контейнерами клиента и API и согласовать префикс /api с настройкой VITE_API_URL=/api у клиента.

3.12 Выводы по разделу

1. Серверная часть реализована на Node.js с использованием фреймворка Express и ORM Prisma для доступа к PostgreSQL 15.

2. Структура REST API включает маршруты аутентификации, пользователя, проектов, шаблонов, изображений и проверки /api/health.

3. Модульная структура сервера и клиента соответствует разделению ответственности; глобальное состояние на клиенте реализовано через React Context и локальное состояние компонентов.

4. Клиентское приложение использует React и Vite; явно объявленные маршруты в App.tsx включают /login, /register, /dashboard, /profile, /editor/:id? и обработчик *.

5. Контейнеризация описана файлами docker-compose.yml, Dockerfile.dev (клиент и сервер) и образом nginx:alpine; данные БД и загрузки пользователей сохраняются именованными томами Docker.

4 Тестирование web-приложения

4.1 Функциональное тестирование

Для проверки работоспособности сценариев было проведено ручное функциональное тестирование интерфейса и взаимодействия с REST API.

Условия: запущены сервер приложения (Node.js, Express) и PostgreSQL (локально или в Docker); клиент открыт в браузере с поддержкой современного JavaScript; тестирование выполнялось вручную по контрольным сценариям без регистрации результатов в системе управления тестами.

Описание шагов и ожидаемых результатов приведено в таблице 4.1.

Таблица 4.1 – Описание тестирования функций web-приложения «Конструктор коллажей»

Функция web-приложения	Описание тестирования	Ожидаемый результат	Фактический результат
1	2	3	4
Регистрация	Страница /register: корректный email, пароль не короче 6 символов, отправка формы	Учётная запись создана; JWT сохраняется; переход к защищённой зоне	Успешно; повторная регистрация с тем же email отклоняется сервером
Авторизация	Страница /login: верные email и пароль существующего пользователя	JWT сохраняется; переход на панель проектов	Успешно; при неверном пароле отображается сообщение об ошибке, доступ не выдаётся
Защита маршрутов	Без входа открыть /dashboard или /editor	Перенаправление на /login	Успешно
Список проектов	После входа открыть /dashboard	Отображаются только проекты текущего пользователя	Успешно
Новый проект	Создать проект с названием и размерами холста	POST /api/projects; переход в /editor/:id с пустым холстом	Успешно; проект появляется в списке
Редактирование холста	Добавить текст, фигуру, изображение; переместить и масштабировать объекты	Объекты отображаются на canvas; нет необработываемых ошибок в консоли	Успешно при объёме сценария учебного проекта

Продолжение таблицы 4.1

1	2	3	4
Сохранение проекта	Изменить коллаж; выполнить сохранение	PUT /api/projects/:id; при повторном открытии состояние восстанавливается	Успешно; превью обновляется при передаче на сервер
Undo / Redo	Изменить холст; выполнить отмену и повтор	Состояние холста соответствует ожидаемому после операций	Успешно
Удаление проекта	Удалить проект с подтверждением в интерфейсе	DELETE успешен; запись исчезает из списка	Успешно
Шаблоны	Открыть список шаблонов; применить шаблон к проекту	Холст заполняется данными шаблона	Успешно
Загрузка файла	Загрузить изображение с устройства в медиатеку	Запись в медиатеке; файл можно вставить на холст	Успешно для поддерживаемых форматов и ограничений размера
Unsplash	Поиск и добавление изображения	Изображение добавляется на холст при заданном UNSPLASH_ACCESS_KEY; при ошибке ключа – предсказуемое сообщение или пустой результат	Успешно при наличии ключа; без ключа – ожидаемое ограничение функции
Экспорт	Экспорт готового коллажа в файл изображения	Файл формируется и скачивается браузером	Успешно
Профиль	Страница /profile: изменить имя; сохранить	Данные профиля обновляются на сервере	Успешно; после перезагрузки страницы отображается новое имя

Итог ручных проверок: для перечисленных сценариев фактическое поведение совпало с ожидаемым при корректной настройке окружения (запущенный API, доступная PostgreSQL, при экспорте через Nginx – валидные сертификаты или прямой доступ к портам dev-серверов).

4.2 Автоматизированное тестирование

Для автоматизированной регрессионной проверки серверной части подготовлена коллекция запросов к REST API (Postman). Полный прогон коллекции выполнялся в автоматическом режиме с помощью Newman при запущенных сервере

приложения и PostgreSQL (локально или в составе Docker Compose). Последовательность включала получение токена после регистрации/входа, проверку защищённых маршрутов с заголовком Authorization: Bearer, загрузку файла multipart и контроль кодов ответа для негативных сценариев. По результатам тестирования все пятнадцать сценариев завершились успешно; расхождений фактических кодов HTTP и тел ответов с ожидаемыми не выявлено. Результаты автоматизированного тестирования REST API представлены в таблице 4.2.

Таблица 4.2 – Результаты автоматизированного тестирования REST API

Код	Проверка	Запрос	Ожидание	Результат
1	2	3	4	5
АТ-01	Активность сервиса и БД	GET /api/health	Код 200; в теле признак успешного подключения к БД	Пройдено
АТ-02	Регистрация пользователя	POST /api/auth/register, тело JSON с полями name, email, password	Код 201 или 200; в ответе token и данные пользователя	Пройдено
АТ-03	Отказ регистрации при дубликате email	POST /api/auth/register с уже существующим email	Код 4xx; пользователь не создаётся повторно	Пройдено
АТ-04	Вход	POST /api/auth/login	Код 200; token при верных учётных данных	Пройдено
АТ-05	Отказ входа	POST /api/auth/login с неверным паролем	Код 401	Пройдено
АТ-06	Текущий пользователь	GET /api/auth/me с заголовком Authorization: Bearer ...	Код 200; профиль соответствует владельцу token	Пройдено
АТ-07	Список проектов	GET /api/projects с Bearer	Код 200; массив проектов только текущего пользователя	Пройдено
АТ-08	Создание проекта	POST /api/projects с Bearer и JSON title, width, height	Код 201; создан проект с id	Пройдено
АТ-09	Чтение проекта	GET /api/projects/:id с Bearer	Код 200; тело содержит canvasData при наличии сохранения	Пройдено

Продолжение таблицы 4.2

1	2	3	4	5
АТ-10	Обновление проекта	PUT /api/projects/:id с Bearer и JSON	Код 200; изменения сохраняются при повторном GET	Пройдено
АТ-11	Удаление проекта	DELETE /api/projects/:id с Bearer	Код 200 или 204; проект отсутствует в списке	Пройдено
АТ-12	Шаблоны	GET /api/templates с Bearer	Код 200; список доступных шаблонов	Пройдено
АТ-13	Загрузка файла	POST /api/images/upload, multipart, поле file, Bearer	Код 201; в ответе данные media	Пройдено
АТ-14	Медиатека	GET /api/images/library с Bearer	Код 200; список загруженных файлов пользователя	Пройдено
АТ-15	Доступ без токена	GET /api/projects без Authorization	Код 401	Пройдено

Таким образом, автоматизированное тестирование API даёт воспроизводимый контроль качества backend-слоя; вместе с ручным функциональным тестированием интерфейса.

4.3 Нагрузочное тестирование

Выполнено нагрузочное испытание точки входа API без авторизации – GET /api/health – как типового обращения с минимальной логикой и проверкой подключения к базе данных. Инструмент: Apache Bench (ab); целевой адрес: <http://127.0.0.1:3000/api/health> при локально запущенном сервере Node.js и доступной PostgreSQL. Параметры запуска: всего 2000 запросов, уровень параллелизма 10 одновременных соединений. Измерение выполнялось на рабочей станции разработчика в условиях локальной сети без учёта задержек WAN и без балансировщика нагрузки; полученные показатели характеризуют производительность данной конфигурации при синтетической однотипной нагрузке. Численные результаты испытания приведены в табл. 4.3.

Таблица 4.3 – Результаты нагрузочного испытания (GET /api/health)

Показатель	Значение
1	2
Число запросов	2000

Продолжение таблицы 4.3

1	2
Параллельность (одновременных соединений)	10
Успешных ответов (HTTP 200), шт.	2000
Неуспешных ответов, шт.	0
Доля успешных ответов, %	100
Общее время выполнения серии запросов, с	2,35
Среднее время ответа на запрос (mean), мс	11,7
Медиана времени ответа, мс	11

По данным табл. 4.3 все 2000 запросов завершились без ошибок (Failed requests: 0), доля успешных ответов – 100 %. Среднее время полного цикла запроса при параллельности 10 составило около 11,7 мс (медиана 11 мс, наиболее долгий ответ 56 мс); пропускная способность – около 851 запроса в секунду при общей длительности серии около 2,35 с. Это указывает на устойчивую обработку синтетической однотипной нагрузки на данном стенде; абсолютные значения зависят от аппаратной конфигурации рабочей станции, версии Node.js и состояния ОС и не переносятся автоматически на промышленную среду. Маршрут не отражает стоимость операций сохранения крупного `canvasData` или выдачи файлов из `/uploads`.

4.4 Выводы по разделу

1. Выполнено ручное функциональное тестирование; фактическое поведение соответствует ожидаемому при корректной настройке окружения (включая ключ `Unsplash` для сценариев поиска стоковых изображений).

2. Выполнено автоматизированное регрессионное тестирование REST API; все сценарии пройдены, работоспособность основных маршрутов подтверждена.

3. Выполнено нагрузочное испытание маршрута `GET /api/health`; зафиксированы показатели времени ответа и пропускной способности для используемой конфигурации стенда.

5 Руководство пользователя

5.1 Руководство для роли «Гость»

Гость — пользователь, который ещё не вошёл в систему. При обращении к адресу приложения по умолчанию маршрутизация перенаправляет на панель проектов /dashboard; так как она защищена, выполняется переход на страницу входа /login. Таким образом, стартовым экраном для гостя является страница авторизации (или страница регистрации, если пользователь открыл её напрямую). На страницах входа и регистрации отображаются пояснения о возможностях сервиса (редактор, библиотека изображений, экспорт и т.д.)

Переход между режимами «гость» и «клиент» выполняется только после успешной регистрации или входа: до этого момента недоступны список коллажей и сохранение проектов на сервере.

5.1.1 Регистрация

Если учётной записи ещё нет, необходимо перейти на страницу /register. Далее следует заполнить поля формы: имя, email, пароль длиной не менее шести символов. Для создания учётной записи нужно нажать кнопку «Зарегистрироваться». Форма регистрации представлена на рисунке 5.1.

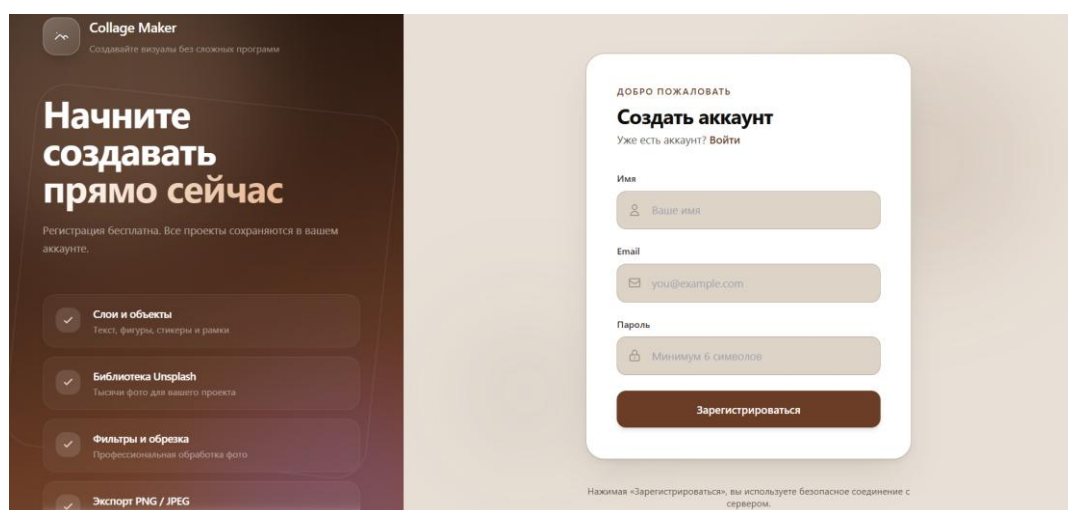


Рисунок 5.1 – Форма регистрации

После успешной регистрации выполняется автоматический вход. Пользователь сразу получает доступ к функциям клиента и перенаправляется на страницу /dashboard.

5.1.2 Доступ к функциям без авторизации

Просмотреть и изменять сохранённые коллажи без входа нельзя: при попытке открыть страницу /dashboard или /profile приложение перенаправляет гостя на /login. Редактор по адресу /editor/ технически может открываться из адресной строки, но сохранение проектов на сервере потребует авторизации. Таким образом,

полноценная работа с проектами начинается после регистрации или входа. Перенаправление на страницу входа изображено на рисунке 5.2.

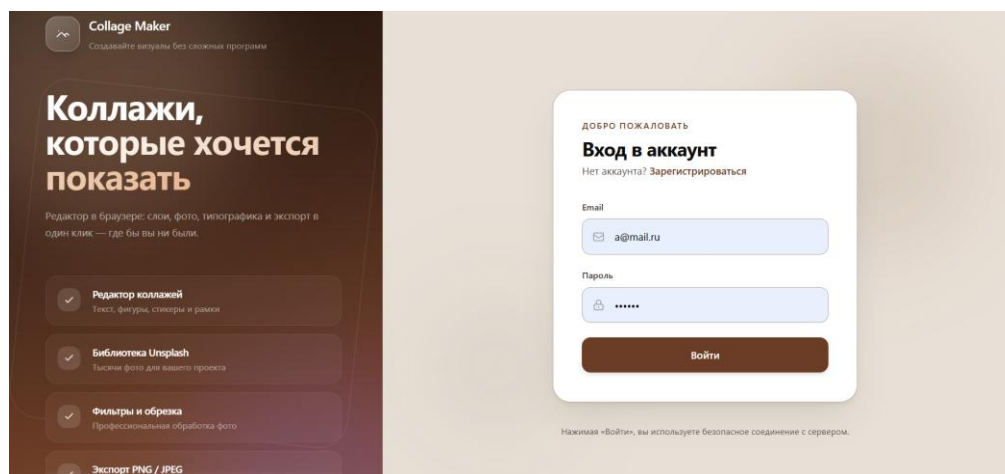


Рисунок 5.2 – Перенаправление на страницу входа

Использование разделов руководства для клиента предполагает, что описанные действия выполняются уже после успешного входа в систему.

5.2 Руководство для роли «Клиент»

Клиент – это авторизованный пользователь. После входа открывается страница /dashboard («Мои проекты»): отображается количество проектов, список карточек коллажей либо подсказка создать первый коллаж. В шапке страницы показываются имя или email, кнопки «Профиль» и «Выйти». Панель проектов представлена на рисунке 5.3.

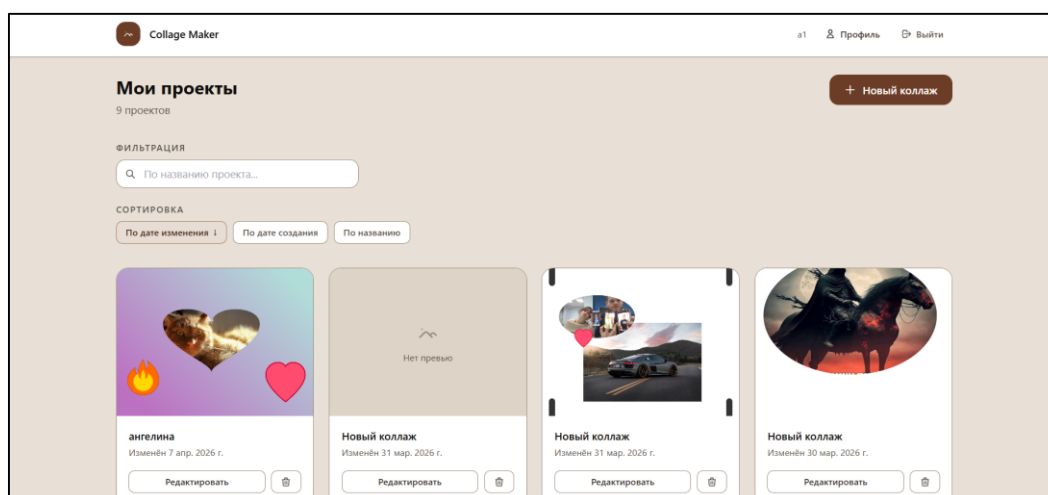


Рисунок 5.3 – Панель проектов («Мои проекты»)

Управление коллажами выполняется с этой страницы: создание нового проекта, переход в редактор, удаление проекта, поиск по названию и сортировка списка.

5.2.1 Аутентификация

Для входа нужно открыть страницу /login. В форме необходимо указать электронную почту и пароль, после чего нажать кнопку «Войти». Форма аутентификации представлена на рисунке 5.4.

Рисунок 5.4 – Форма аутентификации

После успешной авторизации пользователь считается клиентом.

5.2.2 Создание коллажа и редактор

Чтобы начать новый проект, на панели проектов нужно нажать кнопку «Новый коллаж». В появившемся окне задаётся название и размеры холста, далее следует подтвердить создание. Откроется редактор с пустым холстом. Окно создания проекта и интерфейс редактора коллажа представлены на рисунке 5.5.

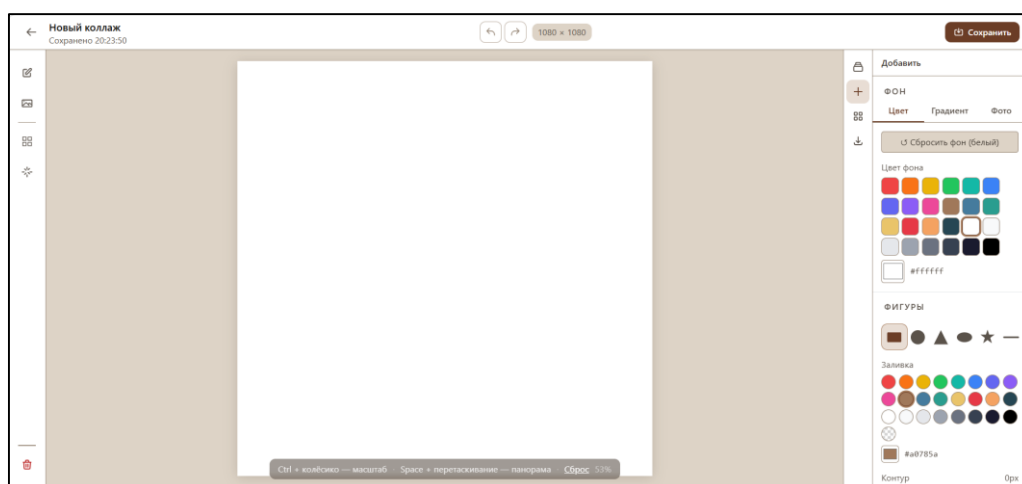


Рисунок 5.5 – Окно создания проекта и интерфейс редактора коллажа

В редакторе доступны добавление текста, изображений (загрузка с устройства или поиск), фигур, стикеров, настройка фона, фильтры, обрезка, работа со слоями, отмена и повтор действий, при необходимости сетка.

5.2.3 Шаблоны, медиатека и экспорт

Для ускорения работы можно открыть панель шаблонов и применить готовый макет к холсту. Загруженные ранее файлы доступны в медиатеке пользователя. Готовый коллаж можно экспортировать в графический файл через интерфейс экспорта (форматы и параметры зависят от реализации клиента). Пример диалога экспорта коллажа виден на рисунке 5.6.

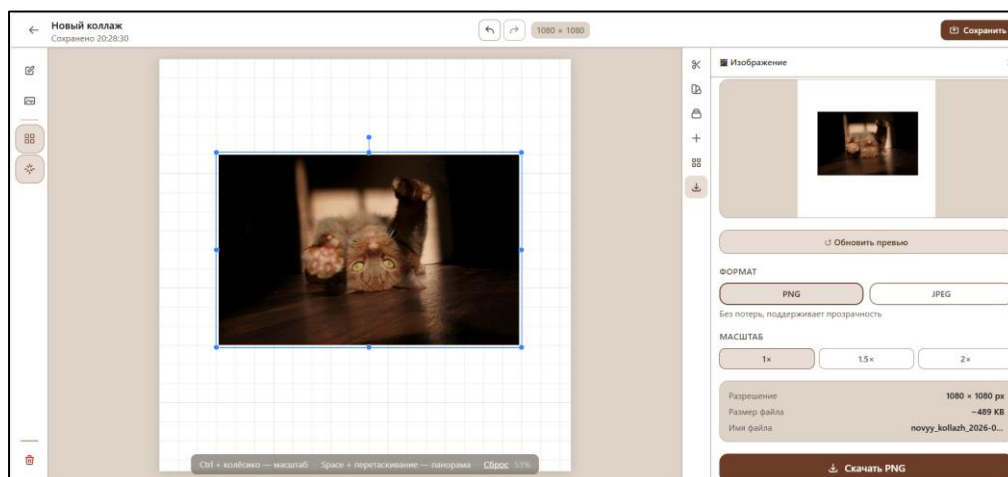


Рисунок 5.6 – Пример диалога экспорта коллажа

После экспорта файл сохраняется на устройство пользователя; сам проект при этом остаётся в списке на /dashboard, пока его не удалят.

5.2.4 Редактирование профиля

Для изменения отображаемого имени нужно нажать в шапке страницы кнопку «Профиль» – откроется страница /profile. Поле email и роль отображаются только для просмотра. Можно отредактировать имя и сохранить изменения. Страница профиля пользователя представлена на рисунке 5.7.

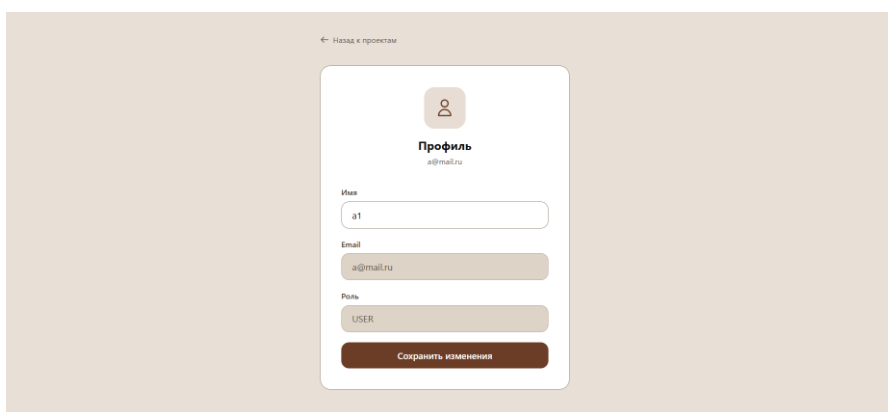


Рисунок 5.7 – Страница профиля пользователя

Выход из системы выполняется только кнопкой «Выйти».

5.2.5 Выход из системы

Чтобы завершить сеанс работы, на панели проектов («Мои проекты») в правой части шапки страницы расположены действия учётной записи: переход в профиль и выход из системы. Кнопка «Выйти» завершает локальную сессию: сохранённый в браузере токен удаляется, доступ к защищённым страницам (/dashboard, /profile, редактор с сохранением на сервере) становится недоступен до повторного входа.

Интерфейс фрагмента шапки с кнопкой «Выйти» представлен на рисунке 5.8.

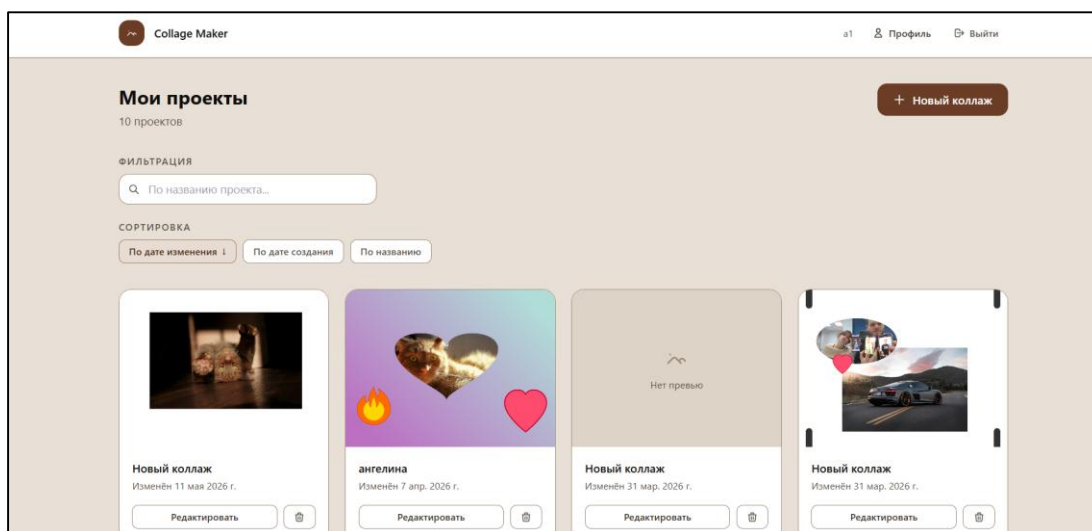


Рисунок 5.8 – Панель проектов: кнопка выхода из системы

После нажатия «Выйти» выполняется переход на страницу входа (/login). Для дальнейшей работы с коллажами потребуется снова ввести email и пароль и выполнить вход в систему.

5.3 Выводы по разделу

В руководстве пользователя описаны действия для ролей «Гость» и «Клиент» web-приложения «Конструктор коллажей»: вход, регистрация, ограничения для неавторизованных пользователей, панель проектов, редактор, шаблоны и экспорт, профиль, выход из системы.

Гость может зарегистрироваться и войти в систему; без авторизации доступ к панели проектов и профилю закрыт перенаправлением на страницу входа.

Клиент может создавать и удалять проекты, работать в редакторе коллажа, пользоваться шаблонами и экспортом, искать и сортировать проекты на /dashboard, изменять имя в профиле и завершать сеанс кнопкой «Выйти».

6 Руководство программиста

6.1 Требования к окружению

Для сборки и запуска клиента и сервера используется среда Node.js и пакетный менеджер npm. При локальном запуске базы данных без контейнера требуется установленный PostgreSQL; при контейнерном варианте база поднимается из образа postgres:15-alpine. Развёртывание и совместный запуск сервисов описаны в docker-compose.yml; для работы с Compose необходимы Docker Engine и подкоманда compose (например Docker Desktop под Windows). Перечень компонентов окружения для сборки и запуска приведён в таблице 6.1.

Таблица 6.1 – Программное окружение для разработки и запуска

Компонент	Версия / требование	Назначение
Node.js	20.x LTS	Среда выполнения сервера и сборки клиента
npm	входит в состав Node.js	Установка зависимостей, скрипты package.json
PostgreSQL	15	СУБД приложения
Docker и Docker Compose	актуальный стабильный релиз	Контейнеры db, server, client, nginx
Браузер	с поддержкой современного JavaScript	Отладка SPA-клиента

Таблица 6.1 задаёт минимальный набор инструментов; версии следует согласовать с используемой в проекте конфигурацией package.json и образами Docker.

6.2 Переменные окружения

Конфиденциальные значения (пароли БД, JWT_SECRET, ключ Unsplash) не включаются в систему контроля версий и задаются в файлах .env на машине разработчика или на сервере развёртывания. В корне репозитория файл .env используется при запуске docker compose; переменные подставляются в docker-compose.yml (имена пользователя и базы, секрет JWT и др.). В каталоге server файл .env задаёт строку DATABASE_URL для режима без Docker и JWT_SECRET; при необходимости подключения поиска изображений на сервере указывается UNSPLASH_ACCESS_KEY. Наименования переменных и место их задания сведены в таблице 6.2.

Таблица 6.2 – Основные переменные окружения

Переменная	Где задаётся	Назначение
1	2	3
DB_USER, DB_PASSWORD, DB_NAME	Корневой .env для Docker	Учётная запись и имя базы PostgreSQL в контейнере

Продолжение таблицы 6.2

1	2	3
JWT_SECRET, JWT_EXPIRES_IN	Корневой .env / окружение server в Compose	Подпись и срок действия JWT
DATABASE_URL	server/.env без Docker	Строка подключения Prisma к PostgreSQL
UNSPLASH_ACCESS_KEY	server/.env	Доступ к API Unsplash с сервера
PORT	При необходимости в окружении server	Порт HTTP API (по умолчанию 3000)
VITE_API_URL, API_PROXY_TARGET	Окружение контейнера client в Compose	Префикс API и цель прокси Vite внутри Docker

Использование переменных из табл. 6.2 должно учитывать то, что секреты не передаются в открытых репозиториях и задаются локально. Пример содержимого server/.env для автономного запуска сервера без Docker приведён в листинге 6.1.

```
DATA-
BASE_URL="postgresql://USER:PASSWORD@localhost:5432/collage_db?schem
a=public"

JWT_SECRET="случайная_длинная_строка"

JWT_EXPIRES_IN="7d"

UNSPLASH_ACCESS_KEY="ключ_при_необходимости"
```

Листинг 6.1 – Пример файла server/.env

Фрагмент в листинге 6.1 иллюстрирует типичный состав параметров; значения JWT_SECRET и ключей подбираются индивидуально и не должны совпадать с приведёнными условными строками.

6.3 Запуск без Docker

Последовательность действий:

1. Установить PostgreSQL 15, создать базу данных и пользователя с правами на неё.
2. В каталоге server создать файл .env по образцу листинга 6.1; выполнить npm install и npm run dev. При старте выполняется синхронизация схемы Prisma с базой (prisma db push в режиме разработки). API слушает порт 3000.
3. В каталоге client выполнить npm install и npm run dev. Интерфейс доступен по адресу http://localhost:5173. Запросы путей /api и /uploads проксируются на backend; соответствующий фрагмент конфигурации Vite показан в листинге 6.2 (переменная API_PROXY_TARGET задаёт цель прокси в Docker; локально используется localhost:3000).

```
const proxyTarget = process.env.API_PROXY_TARGET ||  
'http://localhost:3000';  
// ...  
proxy: { '/api': { target: proxyTarget, changeOrigin: true },  
  '/uploads': { target: proxyTarget, changeOrigin: true } }
```

Листинг 6.2 – Фрагмент vite.config.ts

Настройки листинга 6.2 обеспечивают прозрачную маршрутизацию API и статических загрузок на один backend-порт при разработке.

4. Проверка: запрос GET `http://localhost:3000/api/health` должен вернуть JSON с признаком успешного подключения к базе данных. Успешный ответ представлен на рисунке 6.1.

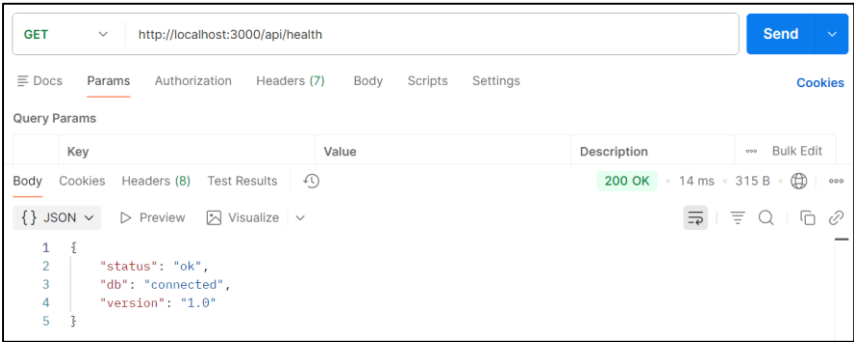


Рисунок 6.1 – Успешный ответ эндпоинта /api/health

Скриншот по рисунку 6.1 служит подтверждением готовности API и базы перед проверкой через пользовательский интерфейс.

6.4 Запуск с Docker Compose

В корне репозитория нужно выполнить команду `docker compose up --build`. Состав сервисов (db, server, client, nginx), тома и проброс портов определены в `docker-compose.yml`. Контейнер сервера ожидает готовности базы по `healthcheck`; загрузки пользователей монтируются в том `uploads_data`. Соответствие портов хоста сервисам приведено в таблице 6.3.

Таблица 6.3 – Проброс портов при запуске через Docker Compose

Порт узла	Сервис в Compose	Назначение
5432	db	PostgreSQL (прямой доступ при отладке БД)
3000	server	REST API Node.js
5173	client	Dev-сервер Vite (SPA)
80	nginx	HTTP с редиректом на HTTPS
443	nginx	HTTPS, обратный прокси к client и server

Табл. 6.3 показывает, по каким портам узла доступны отдельные сервисы; для пользователя через браузер основным входом при настроенном TLS выступает nginx на 443.

Для работы HTTPS в nginx в каталог nginx/ssl должны быть файлы cert.pem и key.pem. Обращение к приложению через nginx: https://localhost. Фрагмент правил проксирования к сервисам по именам контейнеров приведён в листинге 6.3.

```
location /api/ {
    proxy_pass http://server:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto https;}
location / {
    proxy_pass http://client:5173;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "Upgrade";}
```

Листинг 6.3 – Фрагмент nginx/default.conf (проксирование на сервисы Compose)

Указанные в листинге 6.3 директивы location определяют, что API и загрузки направляются на контейнер server, а корень сайта – на dev-сервер Vite в контейнере client; полный файл включает также настройки SSL и редирект с порта 80.

6.5 Проверка работоспособности

Рекомендуемая последовательность проверки после запуска:

1. Убедиться в ответе GET .../api/health (подключение к БД в теле ответа).
2. Зарегистрировать пользователя и выполнить вход через клиентское приложение.
3. Создать проект коллажа, внести изменения в редакторе и выполнить сохранение.
4. Для контейнерного варианта дополнительно убедиться, что контейнеры db и server в состоянии healthy / работают без перезапусков в цикле.

Экран входа или регистрации представлен на рисунке 6.2.

Рисунок 6.2 – Регистрация или вход в приложении после развёртывания

Работа редактора после сохранения проекта представлена на рисунке 6.3.

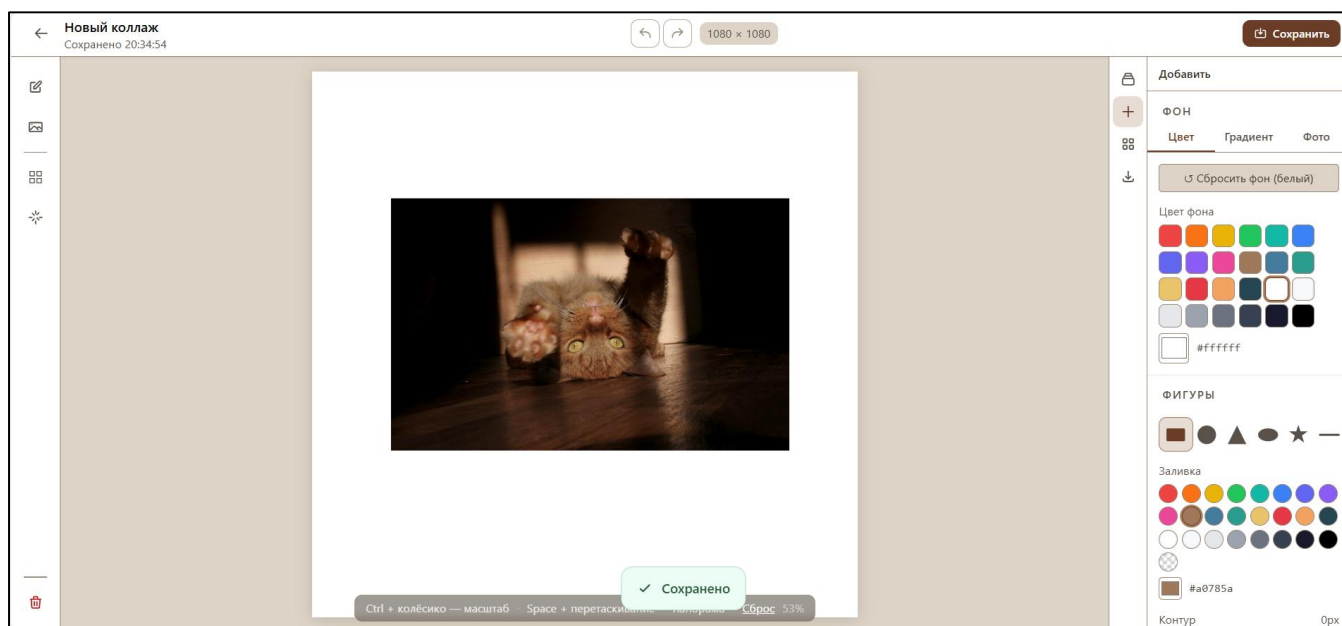


Рисунок 6.3 – Редактор коллажа с сохранённым проектом

Приведённые на рисунках 6.2–6.3 материалы подтверждают прохождение сквозного сценария от учётной записи до сохранённого коллажа.

6.6 Выводы по разделу

1. Развёртывание может выполняться локально (Node.js, PostgreSQL, два процесса `npm run dev`) или целиком в Docker Compose с обратным прокси Nginx и TLS; состав переменных окружения сведён в таблице 6.2, порты – в таблице 6.3.

2. Минимальная проверка включает health-check API, прохождение сценария регистрации и сохранения коллажа.

Заключение

В ходе курсового проекта реализован интернет-сервис «Конструктор коллажей» для создания и хранения коллажей в браузере.

Количество ролей предметной области: 2 (Гость и Клиент).

Количество функций по таблицам 2.2 и 2.3: 2 варианта использования для Гостя и 19 для Клиента, всего 21 функция.

Количество таблиц в базе данных: 5 (users, projects, collage_elements, templates, media).

Особенности архитектуры: одностраничное приложение на React и Vite; REST API на Node.js и Express; аутентификация JWT; ORM Prisma и PostgreSQL; статическая раздача загрузок; внешний сервис Unsplash; контейнеризация Docker Compose и обратный прокси Nginx с TLS.

Объём исходного кода: сервер около 974 строк, клиент около 5332 строк, всего около 6306 строк.

Тестирование: выполнено ручное функциональное, автоматизированное регрессионное тестирование API, нагрузочное испытание точки GET /api/health.

Список используемых источников

- 1 React 18.2. Official Documentation [Электронный ресурс] / Режим доступа: <https://react.dev/> – Дата доступа: 06.04.2026.
- 2 Vite 5.1. Documentation [Электронный ресурс] / Режим доступа: <https://vitejs.dev/> – Дата доступа: 18.04.2026.
- 3 Node.js 20 Documentation [Электронный ресурс] / Режим доступа: <https://nodejs.org/docs/latest-v20.x/api/> – Дата доступа: 12.03.2026.
- 4 Express.js 4.18. Documentation [Электронный ресурс] / Режим доступа: <https://expressjs.com/en/4x/api.html> – Дата доступа: 14.03.2026.
- 5 PostgreSQL 15 Documentation [Электронный ресурс] / Режим доступа: <https://www.postgresql.org/docs/15/> – Дата доступа: 22.03.2026.
- 6 Prisma ORM 5.10. Documentation [Электронный ресурс] / Режим доступа: <https://www.prisma.io/docs> – Дата доступа: 18.03.2026.
- 7 Docker Documentation [Электронный ресурс] / Режим доступа: <https://docs.docker.com/> – Дата доступа: 04.05.2026.
- 8 Docker Compose Documentation [Электронный ресурс] / Режим доступа: <https://docs.docker.com/compose/> – Дата доступа: 04.05.2026.
- 9 Nginx Documentation [Электронный ресурс] / Режим доступа: <https://nginx.org/en/docs/> – Дата доступа: 22.04.2026.
- 10 HTTP Secure (HTTPS) [Электронный ресурс] / Режим доступа: <https://developer.mozilla.org/en-US/docs/Glossary/HTTPS> – Дата доступа: 25.04.2026.
- 11 Canva [Электронный ресурс] / Режим доступа: <https://www.canva.com/> – Дата доступа: 03.03.2026.
- 12 PhotoCollage.com [Электронный ресурс] / Режим доступа: <https://www.photocollage.com/> – Дата доступа: 05.03.2026.
- 13 Progif [Электронный ресурс] / Режим доступа: <https://progif.ru/> – Дата доступа: 08.03.2026.
- 14 jsonwebtoken 9.0.2 [Электронный ресурс] / Режим доступа: <https://www.npmjs.com/package/jsonwebtoken> – Дата доступа: 25.03.2026.
- 15 bcryptjs 2.4.3 [Электронный ресурс] / Режим доступа: <https://www.npmjs.com/package/bcryptjs> – Дата доступа: 28.03.2026.
- 16 Multer 2.1.1 [Электронный ресурс] / Режим доступа: <https://www.npmjs.com/package/multer> – Дата доступа: 02.04.2026.
- 17 Axios 1.6.7 Documentation [Электронный ресурс] / Режим доступа: <https://axios-http.com/docs/intro> – Дата доступа: 10.04.2026.
- 18 React Router DOM 6.22.1 Documentation [Электронный ресурс] / Режим доступа: <https://reactrouter.com/> – Дата доступа: 14.04.2026.
- 19 Unsplash API Documentation [Электронный ресурс] / Режим доступа: <https://unsplash.com/documentation> – Дата доступа: 26.04.2026.
- 20 Cross-Origin Resource Sharing (CORS) [Электронный ресурс] / Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS> – Дата доступа: 30.04.2026.
- 21 Fabric.js Documentation [Электронный ресурс] / Режим доступа: <http://fabricjs.com/> – Дата доступа: 15.05.2026.

ПРИЛОЖЕНИЕ А

Листинг файла server/src/index.ts

```

import express, { Request, Response, NextFunction } from 'express';
import cors from 'cors';
import path from 'path';
import { PrismaClient } from '@prisma/client';

const app = express();
const prisma = new PrismaClient();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json({ limit: '50mb' }));
app.use(express.urlencoded({ limit: '50mb', extended: true }));

// Статика для загруженных файлов
app.use('/uploads', express.static(path.join(process.cwd(), 'uploads')));

import authRoutes from './routes/auth.routes';
import userRoutes from './routes/user.routes';
import projectRoutes from './routes/project.routes';
import templateRoutes from './routes/template.routes';
import imageRoutes from './routes/image.routes';

// Логирование всех входящих запросов
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});

// API Маршруты
app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);
app.use('/api/projects', projectRoutes);
app.use('/api/templates', templateRoutes);
app.use('/api/images', imageRoutes);

app.get('/api/health', async (req, res, next) => {
  try {
    // Простой запрос к БД для проверки соединения
    await prisma.$queryRaw`SELECT 1`;
    res.status(200).json({ status: 'ok', db: 'connected', version: '1.0' });
  } catch (error) {
    console.error('Database connection failed', error);
    // Передаем ошибку в глобальный обработчик
    next(error);
  }
});

```

```
// Базовый глобальный обработчик ошибок
app.use((err: Error, req: Request, res: Response, next:
NextFunction) => {
  console.error('\x1b[31m[Global Error Handler]\x1b[0m',
err.stack || err.message);
  res.status(500).json({ error: 'Internal Server Error', mes-
sage: err.message });
});

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

ПРИЛОЖЕНИЕ Б

Листинг файла server/prisma/schema.prisma

```

generator client {
  provider      = "prisma-client-js"
  binaryTargets = ["native", "linux-musl", "linux-musl-openssl-
3.0.x"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ===== ENUMS =====
enum Role {
  GUEST
  USER
  ADMIN
}

enum MediaSource {
  UPLOAD
  UNSPLASH
}

enum ElementType {
  IMAGE
  TEXT
  SHAPE
  STICKER
}

// ===== MODELS =====
model User {
  id          String      @id @default(uuid())
  email       String?     @unique
  passwordHash String?
  name        String?
  role        Role        @default(GUEST)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  projects    Project[]
  templates   Template[]
  media        Media[]

  @@map("users")
}

model Project {
  id          String      @id @default(uuid())
  title       String

```

```

    canvasData      Json
    thumbnailUrl    String?
    userId          String
    user            User          @relation(fields: [userId], references:
[id], onDelete: Cascade)
    createdAt       DateTime @default(now())
    updatedAt       DateTime @updatedAt

    elements        CollageElement[]

    @@map("projects")
}

model CollageElement {
    id              String          @id @default(uuid())
    type            ElementType
    data            Json
    position        Json
    layer           Int
    projectId       String
    project         Project          @relation(fields: [projectId], references:
[id], onDelete: Cascade)
    createdAt       DateTime @default(now())
    updatedAt       DateTime @updatedAt

    @@map("collage_elements")
}

model Template {
    id              String          @id @default(uuid())
    name            String
    description      String?
    thumbnailUrl    String?
    canvasData      Json
    category         String          @default("Other")
    isPublic        Boolean @default(false)
    userId          String?
    user            User?           @relation(fields: [userId], references:
[id], onDelete: Cascade)
    createdAt       DateTime @default(now())
    updatedAt       DateTime @updatedAt

    @@map("templates")
}

model Media {
    id              String          @id @default(uuid())
    url             String
    filename        String?
    mimeType        String?
    size            Int?
    width           Int?
    height          Int?

```

```
    source      MediaSource @default(UPLOAD)
    authorName  String?
    authorUrl   String?
    userId      String
    user        User         @relation(fields: [userId], references:
[id], onDelete: Cascade)
    createdAt   DateTime     @default(now())

    @@map("media")
}
```

ПРИЛОЖЕНИЕ В

Листинг файла server/src/middlewares/auth.middleware.ts

```
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';

export interface AuthRequest extends Request {
  user?: { id: string };
}

export const protect = (req: AuthRequest, res: Response, next: NextFunction) => {
  let token;

  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'fallback_secret') as { id: string };

      // Добавляем ID пользователя в объект запроса
      req.user = { id: decoded.id };
      next();
    } catch (error) {
      console.error(error);
      return res.status(401).json({ message: 'Не авторизован, неверный токен' });
    }
  }

  if (!token) {
    return res.status(401).json({ message: 'Не авторизован, нет токена' });
  }
};

// ===== server/src/controllers/auth.controller.ts =====

import { Request, Response } from 'express';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { PrismaClient } from '@prisma/client';
import { AuthRequest } from '../middlewares/auth.middleware';

const prisma = new PrismaClient();

const generateToken = (id: string) => {
  // Должен совпадать с секретом в middlewares/auth.middleware.ts
  return jwt.sign({ id }, process.env.JWT_SECRET || 'fallback_secret', {
    expiresIn: (process.env.JWT_EXPIRES_IN || '7d') as jwt.SignOptions['expiresIn'],
  });
};
```

```

    });
};

export const register = async (req: Request, res: Response) => {
  try {
    const { name, email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ message: 'Email и пароль обязательны' });
    }

    // Простая валидация
    if (password.length < 6) {
      return res.status(400).json({ message: 'Пароль должен быть не менее 6 символов' });
    }

    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email)) {
      return res.status(400).json({ message: 'Некорректный формат email' });
    }

    const userExists = await prisma.user.findUnique({ where: { email } });
    if (userExists) {
      return res.status(400).json({ message: 'Пользователь с таким email уже существует' });
    }

    const salt = await bcrypt.genSalt(10);
    const passwordHash = await bcrypt.hash(password, salt);

    const user = await prisma.user.create({
      data: {
        name,
        email,
        passwordHash,
        role: 'USER',
      },
    });

    res.status(201).json({
      id: user.id,
      name: user.name,
      email: user.email,
      role: user.role,
      token: generateToken(user.id),
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Ошибка сервера при регистрации' });
  }
};

```

```

    }
  };

export const login = async (req: Request, res: Response) => {
  try {
    const { email, password } = req.body;

    const user = await prisma.user.findUnique({ where: { email } });
    if (!user || !user.passwordHash) {
      return res.status(401).json({ message: 'Неверный email или пароль' });
    }

    const isMatch = await bcrypt.compare(password, user.passwordHash);
    if (!isMatch) {
      return res.status(401).json({ message: 'Неверный email или пароль' });
    }

    res.json({
      id: user.id,
      name: user.name,
      email: user.email,
      role: user.role,
      token: generateToken(user.id),
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Ошибка сервера при входе' });
  }
};

export const getMe = async (req: AuthRequest, res: Response) => {
  try {
    const user = await prisma.user.findUnique({
      where: { id: req.user?.id },
      select: { id: true, name: true, email: true, role: true, createdAt: true },
    });

    if (!user) {
      return res.status(404).json({ message: 'Пользователь не найден' });
    }

    res.json(user);
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Ошибка сервера' });
  }
};

```

```
// ===== server/src/controllers/user.controller.ts =====

import { Response } from 'express';
import { PrismaClient } from '@prisma/client';
import { AuthRequest } from '../middlewares/auth.middleware';

const prisma = new PrismaClient();

export const updateProfile = async (req: AuthRequest, res: Response)
=> {
  try {
    const { name } = req.body;

    const updatedUser = await prisma.user.update({
      where: { id: req.user?.id },
      data: { name },
      select: { id: true, name: true, email: true, role: true, up-
datedAt: true }
    });

    res.json(updatedUser);
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Ошибка обновления профиля' });
  }
};

// ===== server/src/routes/auth.routes.ts =====

import { Router } from 'express';
import { register, login, getMe } from '../controllers/auth.controller';
import { protect } from '../middlewares/auth.middleware';

const router = Router();

router.post('/register', register);
router.post('/login', login);
router.get('/me', protect, getMe);

export default router;

// ===== server/src/routes/user.routes.ts =====

import { Router } from 'express';
import { updateProfile } from '../controllers/user.controller';
import { protect } from '../middlewares/auth.middleware';

const router = Router();

router.put('/profile', protect, updateProfile);

export default router;
```

ПРИЛОЖЕНИЕ Г

Листинг файла server/src/controllers/project.controller.ts

```
import { Response } from 'express';
import { PrismaClient } from '@prisma/client';
import { AuthRequest } from '../../middlewares/auth.middleware';

const prisma = new PrismaClient();

// GET /api/projects
export const getProjects = async (req: AuthRequest, res: Response)
=> {
  try {
    const userId = req.user?.id;
    if (!userId) return res.status(401).json({ message:
'Unauthorized' });

    const projects = await prisma.project.findMany({
      where: { userId },
      orderBy: { updatedAt: 'desc' },
    });

    res.json(projects);
  } catch (error) {
    console.error('Error fetching projects:', error);
    res.status(500).json({ message: 'Internal server error' });
  }
};

// GET /api/projects/:id
export const getProjectById = async (req: AuthRequest, res:
Response) => {
  try {
    const { id } = req.params;
    const userId = req.user?.id;

    if (!userId) return res.status(401).json({ message:
'Unauthorized' });

    const project = await prisma.project.findUnique({
      where: { id },
    });

    if (!project) {
      return res.status(404).json({ message: 'Project not found' });
    }

    if (project.userId !== userId) {
      return res.status(403).json({ message: 'Forbidden: You do not
own this project' });
    }

    res.json(project);
  }
};
```

```

    } catch (error) {
      console.error('Error fetching project:', error);
      res.status(500).json({ message: 'Internal server error' });
    }
  };

// POST /api/projects
export const createProject = async (req: AuthRequest, res: Response)
=> {
  try {
    const { title, width, height } = req.body;
    const userId = req.user?.id;

    if (!userId) return res.status(401).json({ message:
'Unauthorized' });

    const newProject = await prisma.project.create({
      data: {
        title: title || 'Новый проект',
        userId,
        canvasData: { width: width || 1080, height: height || 1080,
objects: [] },
      },
    });

    res.status(201).json(newProject);
  } catch (error) {
    console.error('Error creating project:', error);
    res.status(500).json({ message: 'Internal server error' });
  }
};

// PUT /api/projects/:id
export const updateProject = async (req: AuthRequest, res: Response)
=> {
  try {
    const { id } = req.params;
    const { title, canvasData, thumbnailUrl } = req.body;
    const userId = req.user?.id;

    if (!userId) return res.status(401).json({ message:
'Unauthorized' });

    // Проверяем принадлежность проекта
    const existingProject = await prisma.project.findUnique({ where:
{ id } });
    if (!existingProject) {
      return res.status(404).json({ message: 'Project not found' });
    }
    if (existingProject.userId !== userId) {
      return res.status(403).json({ message: 'Forbidden' });
    }
  }
};

```

```

    const updatedData: any = {};
    if (title !== undefined) updatedData.title = title;
    if (canvasData !== undefined) updatedData.canvasData =
canvasData;
    if (thumbnailUrl !== undefined) updatedData.thumbnailUrl =
thumbnailUrl;

    const updatedProject = await prisma.project.update({
      where: { id },
      data: updatedData,
    });

    res.json(updatedProject);
  } catch (error) {
    console.error('Error updating project:', error);
    res.status(500).json({ message: 'Internal server error' });
  }
};

// DELETE /api/projects/:id
export const deleteProject = async (req: AuthRequest, res: Response)
=> {
  try {
    const { id } = req.params;
    const userId = req.user?.id;

    if (!userId) return res.status(401).json({ message:
'Unauthorized' });

    // Проверяем принадлежность
    const existingProject = await prisma.project.findUnique({ where:
{ id } });
    if (!existingProject) {
      return res.status(404).json({ message: 'Project not found' });
    }
    if (existingProject.userId !== userId) {
      return res.status(403).json({ message: 'Forbidden' });
    }

    await prisma.project.delete({ where: { id } });

    res.json({ message: 'Project deleted successfully' });
  } catch (error) {
    console.error('Error deleting project:', error);
    res.status(500).json({ message: 'Internal server error' });
  }
};

```

ПРИЛОЖЕНИЕ Д

Листинг файла server/src/controllers/template.controller.ts

```
import { Response } from 'express';
import { PrismaClient } from '@prisma/client';
import { AuthRequest } from '../middlewares/auth.middleware';

const prisma = new PrismaClient();

// GET /api/templates?category=&scope=my|public|all
export const getTemplates = async (req: AuthRequest, res: Response)
=> {
  try {
    const userId = req.user?.id;
    const { category, scope = 'all' } = req.query as Record<string,
string>;

    const where: any = {};
    if (category) where.category = category;

    if (scope === 'my' && userId) {
      where.userId = userId;
    } else if (scope === 'public') {
      where.isPublic = true;
    } else {
      // all: публичные + свои
      where.OR = [{ isPublic: true }, ...(userId ? [{ userId } ] :
[])];
    }

    const templates = await prisma.template.findMany({
      where,
      orderBy: { createdAt: 'desc' },
      select: {
        id: true, name: true, thumbnailUrl: true,
        category: true, userId: true, createdAt: true,
      } as any,
    });

    res.json(templates);
  } catch (error) {
    console.error('getTemplates error:', error);
    res.status(500).json({ message: 'Internal server error' });
  }
};

// GET /api/templates/:id
export const getTemplateById = async (req: AuthRequest, res:
Response) => {
  try {
    const { id } = req.params;
    const template = await prisma.template.findUnique({ where: { id
} });
  }
};
```

```

    if (!template) return res.status(404).json({ message: 'Template
not found' });
    // Публичные доступны всем, приватные - только владельцу
    if (!(template as any).isPublic && template.userId !==
req.user?.id) {
        return res.status(403).json({ message: 'Forbidden' });
    }
    res.json(template);
} catch (error) {
    res.status(500).json({ message: 'Internal server error' });
}
};

// POST /api/templates
export const createTemplate = async (req: AuthRequest, res:
Response) => {
    try {
        const userId = req.user?.id;
        if (!userId) return res.status(401).json({ message:
'Unauthorized' });

        const { name, description, canvasData, thumbnailUrl, category,
isPublic } = req.body;
        if (!name || !canvasData) return res.status(400).json({ message:
'name и canvasData обязательны' });

        const template = await prisma.template.create({
            data: {
                name,
                canvasData,
                thumbnailUrl: thumbnailUrl ?? null,
                userId,
            } as any,
        });

        res.status(201).json(template);
    } catch (error) {
        console.error('createTemplate error:', error);
        res.status(500).json({ message: 'Internal server error' });
    }
};

// DELETE /api/templates/:id
export const deleteTemplate = async (req: AuthRequest, res:
Response) => {
    try {
        const { id } = req.params;
        const userId = req.user?.id;
        if (!userId) return res.status(401).json({ message:
'Unauthorized' });

        const template = await prisma.template.findUnique({ where: { id
} });

```

```
    if (!template) return res.status(404).json({ message: 'Not
found' });
    if (template.userId !== userId) return res.status(403).json({
message: 'Forbidden' });

    await prisma.template.delete({ where: { id } });
    res.json({ message: 'Deleted' });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
};
```

ПРИЛОЖЕНИЕ Е

Листинг файла docker-compose.yml (корень репозитория)

```
version: '3.8'

services:
  db:
    image: postgres:15-alpine
    container_name: mcollage_db
    restart: always
    environment:
      POSTGRES_USER: ${DB_USER:-postgres}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-postgres}
      POSTGRES_DB: ${DB_NAME:-collage_db}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-postgres} -d ${DB_NAME:-collage_db}"]
      interval: 5s
      timeout: 5s
      retries: 10

  server:
    build:
      context: ./server
      dockerfile: Dockerfile.dev
    container_name: mcollage_server
    restart: unless-stopped
    volumes:
      - ./server/src:/app/src
      - ./server/prisma:/app/prisma
      - ./server/nodemon.json:/app/nodemon.json
      - /app/node_modules
      - uploads_data:/app/uploads
    ports:
      - "3000:3000"
    env_file:
      - .env
    environment:
      DATABASE_URL: postgresql://${DB_USER:-postgres}:${DB_PASSWORD:-postgres}@db:5432/${DB_NAME:-collage_db}?schema=public
      JWT_SECRET: ${JWT_SECRET:-supersecret}
      JWT_EXPIRES_IN: ${JWT_EXPIRES_IN:-7d}
      PORT: ${PORT:-3000}
    depends_on:
      db:
        condition: service_healthy
    command: npm run dev
```

```

client:
  build:
    context: ./client
    dockerfile: Dockerfile.dev
  container_name: mcollage_client
  restart: unless-stopped
  volumes:
    - ./client/src:/app/src
    - ./client/index.html:/app/index.html
    - ./client/vite.config.ts:/app/vite.config.ts
    - ./client/tsconfig.json:/app/tsconfig.json
    - /app/node_modules
  ports:
    - "5173:5173"
  environment:
    - VITE_API_URL=/api
    # Vite внутри контейнера должен проксировать на сервис server,
a не на localhost
    - API_PROXY_TARGET=http://server:3000
  command: npm run dev

nginx:
  image: nginx:alpine
  container_name: mcollage_nginx
  restart: always
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
    - ./nginx/ssl:/etc/nginx/ssl:ro
  depends_on:
    - client
    - server

volumes:
  postgres_data:
  uploads_data:

# ===== server/Dockerfile.dev =====

FROM node:20-alpine

RUN apk add --no-cache openssl

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

```

```

RUN npx prisma generate

CMD ["npm", "run", "dev"]

# ===== client/Dockerfile.dev =====

FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

CMD ["npm", "run", "dev"]

# ===== nginx/default.conf =====

server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name _;
    client_max_body_size 50m;

    ssl_certificate /etc/nginx/ssl/cert.pem;
    ssl_certificate_key /etc/nginx/ssl/key.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers on;

    location /api/ {
        proxy_pass http://server:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_cache_bypass $http_upgrade;
    }

    location /uploads/ {
        proxy_pass http://server:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

```

```
location / {  
    proxy_pass http://client:5173;  
    proxy_http_version 1.1;  
    proxy_set_header Upgrade $http_upgrade;  
    proxy_set_header Connection "Upgrade";  
    proxy_set_header Host $host;  
    proxy_set_header X-Forwarded-Proto https;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
}  
}
```